



Universidade Estadual de Campinas

Enemy Leo used Appeal! It's
super effective! Pikachu fainted!

Pedro Assunção, Pedro Ferreira, Yvens Porto

2026-08-25

1	Contest
2	Mathematics
2.1	Equations
2.2	Recurrences
2.3	Trigonometry
2.4	Geometry
2.5	Derivatives/Integrals
2.6	Sums
2.7	Series
2.8	Probability theory
2.9	Markov chains
3	Data structures
4	Numerical
4.1	Polynomials and recurrences
4.2	Optimization
4.3	Matrices
4.4	Fourier transforms
5	Number theory
5.1	Modular arithmetic
5.2	Primality
5.3	Divisibility
5.4	Fractions
5.5	Pythagorean Triples
5.6	Primes
5.7	Estimates
5.8	Mobius Function
6	Combinatorial
6.1	Permutations
6.2	Partitions and subsets
6.3	General purpose numbers
7	Graph
7.1	Fundamentals
7.2	Network flow
7.3	Matching
7.4	DFS algorithms
7.5	Coloring
7.6	Heuristics
7.7	Trees
7.8	Math
8	Geometry
8.1	Geometric primitives
8.2	Circles
8.3	Polygons
8.4	Misc. Point Set Problems

1	8.5	3D	22
1	9	Strings	23
1	10	Various	24
2	10.1	Intervals	24
2	10.2	Misc. algorithms	25
2	10.3	Dynamic programming	25
2	10.4	Debugging tricks	25
2	10.5	Optimization tricks	25
3	Contest (1)		
3	template.cpp14 lines		
5	#include <bits/stdc++.h> using namespace std;		
5	#define rep(i, a, b) for(int i = a; i < (b); ++i)		
6	#define all(x) begin(x), end(x)		
7	#define sz(x) (int)(x).size()		
8	typedef long long ll;		
8	typedef pair<int, int> pii;		
8	typedef vector<int> vi;		
9	int main() {		
9	cin.tie(0)->sync_with_stdio(0);		
10	cin.exceptions(cin.failbit);		
10	}		
10	.bashrc3 lines		
11	alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++17 \		
11	-fsanitize=undefined,address'		
11	xmodmap -e 'clear lock' -e 'keycode 66=less greater' #caps =<		
11	.vimrc6 lines		
11	set cin aw ai is ts=4 sw=4 tm=50 nu noeb bg=dark ru cul		
11	sy on im jk <esc> im kj <esc> no ; :		
11	" Select region and then type :Hash to hash your selection.		
11	" Useful for verifying that there aren't mistypes.		
11	ca Hash w !cpp -dD -P -fpreprocessed \ tr -d '[:space:]' \		
11	\ md5sum \ cut -c-6		
12	hash.sh3 lines		
12	# Hashes a file, ignoring all whitespace and comments. Use for		
12	# verifying that code was correctly typed.		
14	cpp -dD -P -fpreprocessed tr -d '[:space:]' md5sum cut -c-6		
15	hashCheck.h		
15	Description: Use this for checking the hash.sh.		
16	" ../number-theory/ModPow.h"882b78, 4 lines		
16	// Computes modular inverse. Assumes mod is a prime.		
16	a26 ll modinv(ll n) {		
16	c57 return modpow(n, mod - 2);		
18	882 }		
18	troubleshoot.txt52 lines		
18	Pre-submit:		
19	Write a few simple test cases if sample is not enough.		
19	Are time limits close? If so, generate max cases.		
21	Is the memory usage fine?		

Could anything overflow?
Make sure to submit the right file.

Wrong answer:
Print your solution! Print debug output, as well.
Are you clearing all data structures between test cases?
Can your algorithm handle the whole range of input?
Read the full problem statement again.
Do you handle all corner cases correctly?
Have you understood the problem correctly?
Any uninitialized variables?
Any overflows?
Confusing N and M, i and j, etc.?
Are you sure your algorithm works?
What special cases have you not thought of?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Is your output format correct? (including whitespace)
Rewrite your solution from the start or let a teammate do it.

Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?

Mathematics (2)

2.1 Equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by $x = -b/2a$.

$$\begin{aligned} ax + by &= e \\ cx + dy &= f \end{aligned} \Rightarrow \begin{aligned} x &= \frac{ed - bf}{ad - bc} \\ y &= \frac{af - ec}{ad - bc} \end{aligned}$$

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A'_i}{\det A}$$

where A'_i is A with the i 'th column replaced by b .

2.2 Recurrences

If $a_n = c_1a_{n-1} + \dots + c_ka_{n-k}$, and $r_1, \dots, r_k$ are distinct roots of $x^k - c_1x^{k-1} - \dots - c_k$, there are $d_1, \dots, d_k$ s.t.

$$a_n = d_1r_1^n + \dots + d_kr_k^n.$$

Non-distinct roots r become polynomial factors, e.g. $a_n = (d_1n + d_2)r^n$.

2.3 Trigonometry

$$\begin{aligned}\sin(v + w) &= \sin v \cos w + \cos v \sin w \\ \cos(v + w) &= \cos v \cos w - \sin v \sin w\end{aligned}$$

$$\begin{aligned}\tan(v + w) &= \frac{\tan v + \tan w}{1 - \tan v \tan w} \\ \sin v + \sin w &= 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2} \\ \cos v + \cos w &= 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2}\end{aligned}$$

$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w .

$$\begin{aligned}a \cos x + b \sin x &= r \cos(x - \phi) \\ a \sin x + b \cos x &= r \sin(x + \phi)\end{aligned}$$

where $r = \sqrt{a^2 + b^2}, \phi = \text{atan2}(b, a)$.

2.4 Geometry

2.4.1 Triangles

Side lengths: a, b, c

Semiperimeter: $p = \frac{a + b + c}{2}$
Area: $A = \sqrt{p(p - a)(p - b)(p - c)}$

Circumradius: $R = \frac{abc}{4A}$

Inradius: $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles):
 $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc \left[1 - \left(\frac{a}{b + c} \right)^2 \right]}$$

Law of sines: $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$
Law of cosines: $a^2 = b^2 + c^2 - 2bc \cos \alpha$
Law of tangents: $\frac{a + b}{a - b} = \frac{\tan \frac{\alpha + \beta}{2}}{\tan \frac{\alpha - \beta}{2}}$

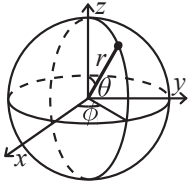
2.4.2 Quadrilaterals

With side lengths a, b, c, d , diagonals e, f , diagonals angle θ , area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin \theta = F \tan \theta = \sqrt{4e^2f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is 180° , $ef = ac + bd$, and $A = \sqrt{(p - a)(p - b)(p - c)(p - d)}$.

2.4.3 Spherical coordinates



$$\begin{aligned}x &= r \sin \theta \cos \phi & r &= \sqrt{x^2 + y^2 + z^2} \\ y &= r \sin \theta \sin \phi & \theta &= \arccos(z / \sqrt{x^2 + y^2 + z^2}) \\ z &= r \cos \theta & \phi &= \text{atan2}(y, x)\end{aligned}$$

2.5 Derivatives/Integrals

$$\begin{aligned}\frac{d}{dx} \arcsin x &= \frac{1}{\sqrt{1 - x^2}} & \frac{d}{dx} \arccos x &= -\frac{1}{\sqrt{1 - x^2}} \\ \frac{d}{dx} \tan x &= 1 + \tan^2 x & \frac{d}{dx} \arctan x &= \frac{1}{1 + x^2} \\ \int \tan ax &= -\frac{\ln |\cos ax|}{a} & \int x \sin ax &= \frac{\sin ax - ax \cos ax}{a^2} \\ \int e^{-x^2} &= \frac{\sqrt{\pi}}{2} \text{erf}(x) & \int xe^{ax} dx &= \frac{e^{ax}}{a^2} (ax - 1)\end{aligned}$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.6 Sums

$$c^a + c^{a+1} + \dots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$$

$$\begin{aligned}1 + 2 + 3 + \dots + n &= \frac{n(n + 1)}{2} \\ 1^2 + 2^2 + 3^2 + \dots + n^2 &= \frac{n(2n + 1)(n + 1)}{6} \\ 1^3 + 2^3 + 3^3 + \dots + n^3 &= \frac{n^2(n + 1)^2}{4} \\ 1^4 + 2^4 + 3^4 + \dots + n^4 &= \frac{n(n + 1)(2n + 1)(3n^2 + 3n - 1)}{30}\end{aligned}$$

2.7 Series

$$\begin{aligned}e^x &= 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots, (-\infty < x < \infty) \\ \ln(1 + x) &= x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \dots, (-1 < x \leq 1) \\ \sqrt{1 + x} &= 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \dots, (-1 \leq x \leq 1) \\ \sin x &= x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots, (-\infty < x < \infty) \\ \cos x &= 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots, (-\infty < x < \infty)\end{aligned}$$

2.8 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x . It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x xp_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y ,

$$V(aX + bY) = a^2V(X) + b^2V(Y).$$

2.8.1 Discrete distributions
Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\text{Bin}(n, p)$, $n = 1, 2, \dots$, $0 \leq p \leq 1$.

$$p(k) = \binom{n}{k} p^k (1 - p)^{n - k}$$

$$\mu = np, \sigma^2 = np(1 - p)$$

$\text{Bin}(n, p)$ is approximately $\text{Po}(np)$ for small p .

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $Fs(p)$, $0 \leq p \leq 1$.

$$p(k) = p(1 - p)^{k-1}, \, k = 1, 2, \dots$$

$$\mu = \frac{1}{p}, \, \sigma^2 = \frac{1 - p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $Po(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda} \frac{\lambda^k}{k!}, k = 0, 1, 2, \dots$$

$$\mu = \lambda, \, \sigma^2 = \lambda$$

2.8.2 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $U(a, b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a + b}{2}, \, \sigma^2 = \frac{(b - a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $Exp(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda}, \, \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ then

$$aX_1 + bX_2 + c \sim \mathcal{N}(\mu_1 + \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

2.9 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. Let $X_1, X_2, \dots$ be a sequence of random variables generated by the Markov process. Then there is a transition matrix $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, and $\mathbf{p}^{(n)} = \mathbf{P}^n \mathbf{p}^{(0)}$ is the probability distribution for X_n (i.e., $p_i^{(n)} = \Pr(X_n = i)$), where $\mathbf{p}^{(0)}$ is the initial distribution.

π is a stationary distribution if $\pi = \pi \mathbf{P}$. If the Markov chain is *irreducible* (it is possible to get to any state from any state), then $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state i . π_j / π_i is the expected number of visits in state j between two visits in state i .

For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, π_i is proportional to node i 's degree.

A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k \rightarrow \infty} \mathbf{P}^k = \mathbf{1}\pi$.

A Markov chain is an A-chain if the states can be partitioned into two sets **A** and **G**, such that all states in **A** are absorbing ($p_{ii} = 1$), and all states in **G** leads to an absorbing state in **A**. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is j , is $a_{ij} = p_{ij} + \sum_{k \in \mathbf{G}} a_{ik} p_{kj}$. The expected time until absorption, when the initial state is i , is $t_i = 1 + \sum_{k \in \mathbf{G}} p_{ki} t_k$.

Data structures (3)

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. To get a map, change null-type. **Time:** $\mathcal{O}(\log N)$

<pre>c4d #include <bits/extc++.h> 0d7 using namespace __gnu_pbds; 4fc template<class T> c20 using Tree = tree<T, null_type, less<T>, rb_tree_tag, 3a1 tree_order_statistics_node_update>; ad0 void example() { c6f Tree<int> t, t2; t.insert(8); 559 auto it = t.insert(10).first; d28 assert(it == t.lower_bound(9)); 969 assert(t.order_of_key(10) == 1); d39 assert(t.order_of_key(11) == 2); 1b7 assert(*t.find_by_order(0) == 8); a60 t.join(t2); // assuming T < T2 or T > T2, merge t2 into t 9ad }</pre>	782797, 16 lines
<pre>bbd90b, 6 lines</pre>	

IndexedSet.h

Description: Similar to OrderStatisticTree, but uses a different library which may be more portable.

<pre>774 #include <ext/pb_ds/assoc_container.hpp> 0d7 using namespace __gnu_pbds; 4fc template<class T> 5eb using IndexedSet = tree<T, null_type, less<T>, rb_tree_tag, 3a1 tree_order_statistics_node_update>; d77092, 7 lines</pre>	
<pre>c4d #include <bits/extc++.h> // To use most bits rather than just the lowest ones: 75f struct chash { // large odd number for C 5d6 const uint64_t C = 11(4e18 * acos(0)) 71; 2cf ll operator()(ll x) const { return __builtin_bswap64(x*C); } cdd }; 911 __gnu_pbds::gp_hash_table<ll,int, chash> h({}, {}, {}, {}, {}, {1<16 });</pre>	

SegmentTree.h

Description: Zero-indexed max-tree. Bounds are inclusive to the left and exclusive to the right. Can be changed by modifying T, f and unit.

Time: $\mathcal{O}(\log N)$

<pre>5ae struct Tree { ef4 typedef int T; cbe static constexpr T unit = INT_MIN; e54 T f(T a, T b) { return max(a, b); } // (any associative fn) 6cd vector<T> s; int n; 3d2 Tree(int n = 0, T def = unit) : s(2*n, def), n(n) {} 6a3 void update(int pos, T val) { 56a for (s[pos += n] = val; pos /= 2;) 326 s[pos] = f(s[pos * 2], s[pos * 2 + 1]); 0e9 } b4c T query(int b, int e) { // query [b, e) 0f9 T ra = unit, rb = unit; fbb for (b += n, e += n; b < e; b /= 2, e /= 2) { e83 if (b % 2) ra = f(ra, s[b++]); 064 if (e % 2) rb = f(s[--e], rb); 561 } cb2 return f(ra, rb); 5b1 } 0f4 };</pre>	0f4bdb, 19 lines
--	------------------

LazySegmentTree.h

Description: Segment tree with ability to add or set values of large intervals, and compute max of intervals. Can be changed to other things. Use with a bump allocator for better performance, and SmallPtr or implicit indices to save memory.

Usage: Node* tr = new Node(v, 0, sz(v));

Time: $\mathcal{O}(\log N)$.

<pre>"../Various/BumpAllocator.h" 34ecf5, 50 lines 1a8 const int inf = 1e9; bf2 struct Node { d65 Node *l = 0, *r = 0; 938 int lo, hi, mset = inf, madd = 0, val = -inf; e9c Node(int lo, int hi):lo(lo),hi(hi){} // Large interval of - inf 7ae Node(vi& v, int lo, int hi) : lo(lo), hi(hi) { cf3 if (lo + 1 < hi) { 7f0 int mid = lo + (hi - lo)/2; c0a l = new Node(v, lo, mid); r = new Node(v, mid, hi); 8da val = max(l->val, r->val); 0ad }</pre>	
---	--

```
cb4      else val = v[lo];
34b    }
2dc    int query(int L, int R) {
7be      if (R <= lo || hi <= L) return -inf;
580      if (L <= lo && hi <= R) return val;
215      push();
8d7      return max(l->query(L, R), r->query(L, R));
f1d    }
b5b    void set(int L, int R, int x) {
d94      if (R <= lo || hi <= L) return;
7e2      if (L <= lo && hi <= R) mset = val = x, madd = 0;
4e6      else {
d2a        push(), l->set(L, R, x), r->set(L, R, x);
8da        val = max(l->val, r->val);
032      }
12a    }
634    void add(int L, int R, int x) {
d94      if (R <= lo || hi <= L) return;
60d      if (L <= lo && hi <= R) {
c27        if (mset != inf) mset += x;
61f        else madd += x;
c61        val += x;
a79      }
4e6      else {
fd7        push(), l->add(L, R, x), r->add(L, R, x);
8da        val = max(l->val, r->val);
1bf      }
aee    }
ecf    void push() {
268      if (!l) {
7f0        int mid = lo + (hi - lo)/2;
fde        l = new Node(lo, mid); r = new Node(mid, hi);
612      }
90f      if (mset != inf)
389        l->set(lo,hi,mset), r->set(lo,hi,mset), mset = inf;
5ce      else if (madd)
ab7        l->add(lo,hi,madd), r->add(lo,hi,madd), madd = 0;
4bc    }
079  };
```

UnionFind.h
Description: Disjoint-set data structure.
Time: $\mathcal{O}(\alpha(N))$

7aa27c, 14 lines

```
ba2 struct UF {
0b0   vi e;
d03   UF(int n) : e(n, -1) {}
3f4   bool sameSet(int a, int b) { return find(a) == find(b); }
84b   int size(int x) { return -e[find(x)]; }
724   int find(int x) { return e[x] < 0 ? x : e[x] = find(e[x]); }
cf0   bool join(int a, int b) {
605     a = find(a), b = find(b);
5c2     if (a == b) return false;
745     if (e[a] > e[b]) swap(a, b);
708     e[a] += e[b]; e[b] = a;
8a6     return true;
33e   }
7aa  };
```

UnionFindRollback.h
Description: Disjoint-set data structure with undo. If undo is not needed, skip st, time() and rollback().
Usage: int t = uf.time(); ...; uf.rollback(t);
Time: $\mathcal{O}(\log(N))$

de4ad0, 21 lines

```
47a struct RollbackUF {
724   vi e; vector<pii> st;
f6f   RollbackUF(int n) : e(n, -1) {}
```

```
84b   int size(int x) { return -e[find(x)]; }
626   int find(int x) { return e[x] < 0 ? x : find(e[x]); }
49f   int time() { return sz(st); }
4db   void rollback(int t) {
314     for (int i = time(); i --> t;)
8d2       e[st[i].first] = st[i].second;
b04     st.resize(t);
30b   }
cf0   bool join(int a, int b) {
605     a = find(a), b = find(b);
5c2     if (a == b) return false;
745     if (e[a] > e[b]) swap(a, b);
bac     st.push_back({a, e[a]});
e6e     st.push_back({b, e[b]});
708     e[a] += e[b]; e[b] = a;
8a6     return true;
6c7   }
de4  };
```

SubMatrix.h
Description: Calculate submatrix sums quickly, given upper-left and lower-right corners (half-open).
Usage: SubMatrix<int> m(matrix);
m.sum(0, 0, 2, 2); // top left 4 elements
Time: $\mathcal{O}(N^2 + Q)$

c59ada, 13 lines

```
4fc template<class T>
3da struct SubMatrix {
94b   vector<vector<T>> p;
fa7   SubMatrix(vector<vector<T>>& v) {
580     int R = sz(v), C = sz(v[0]);
7a2     p.assign(R+1, vector<T>(C+1));
028     rep(r,0,R) rep(c,0,C)
7e0       p[r+1][c+1] = v[r][c] + p[r][c+1] + p[r+1][c] - p[r][c];
e4c   }
031   T sum(int u, int l, int d, int r) {
267     return p[d][r] - p[d][l] - p[u][r] + p[u][l];
b11   }
968  };
```

Matrix.h
Description: Basic operations on square matrices.
Usage: Matrix<int, 3> A;
A.d = {{{{1,2,3}}, {{4,5,6}}, {{7,8,9}}}};
array<int, 3> vec = {1,2,3};
vec = (A^N) * vec;

4da5a2, 26 lines

```
a95 template<class T, int N> struct Matrix {
db4   typedef Matrix M;
c89   array<array<T, N>, N> d{};
02c   M operator*(const M& m) const {
0f2     M a;
1c2     rep(i,0,N) rep(j,0,N)
6fc       rep(k,0,N) a.d[i][k] += d[i][j] * m.d[j][k];
3f5     return a;
956   }
01b   array<T, N> operator*(const array<T, N>& vec) const {
b58     array<T, N> ret{};
a29     rep(i,0,N) rep(j,0,N) ret[i] += d[i][j] * vec[j];
edf     return ret;
bfa   }
70f   M operator^(ll p) const {
5d8     assert(p >= 0);
ccf     M a, b(*this);
72e     rep(i,0,N) a.d[i][i] = 1;
d08     while (p) {
7ae       if (p&1) a = a*b;
e04       b = b*b;
```

```
8b8     p >>= 1;
12e   }
3f5   return a;
5ae   }
4da  };
```

LineContainer.h
Description: Container where you can add lines of the form $kx+m$, and query maximum values at points x . Useful for dynamic programming (“convex hull trick”).
Time: $\mathcal{O}(\log N)$

8ec1c7, 30 lines

```
72c struct Line {
3e2   mutable ll k, m, p;
ca5   bool operator<(const Line& o) const { return k < o.k; }
abf   bool operator<(ll x) const { return p < x; }
7e3  };

781 struct LineContainer : multiset<Line, less<>> {
// (for doubles, use inf = 1/.0, div(a,b) = a/b)
fd2   static const ll inf = LLONG_MAX;
33a   ll div(ll a, ll b) { // floored division
10f     return a / b - ((a ^ b) < 0 && a % b); }
a1c   bool isect(iterator x, iterator y) {
a95     if (y == end()) return x->p = inf, 0;
9cb     if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
591     else x->p = div(y->m - x->m, x->k - y->k);
870     return x->p >= y->p;
2fa   }
a0c   void add(ll k, ll m) {
116     auto z = insert({k, m, 0}), y = z++, x = y;
7b1     while (isect(y, z)) z = erase(z);
141     if (x != begin() && isect(--x, y)) isect(x, y = erase(y));

57d     while ((y = x) != begin() && (--x)->p >= y->p)
774       isect(x, erase(y));
086   }
4ad   ll query(ll x) {
229     assert(!empty());
7d1     auto l = *lower_bound(x);
96a     return l.k * x + l.m;
d21   }
577  };
```

Treap.h
Description: A short self-balancing tree. It acts as a sequential container with log-time splits/joins, and is easy to augment with additional data.
Time: $\mathcal{O}(\log N)$

1754b4, 53 lines

```
bf2 struct Node {
d65   Node *l = 0, *r = 0;
03a   int val, y, c = 1;
fa8   Node(int val) : val(val), y(rand()) {}
86d   void recalc();
daa  };

099 int cnt(Node* n) { return n ? n->c : 0; }
678 void Node::recalc() { c = cnt(l) + cnt(r) + 1; }

7bb template<class F> void each(Node* n, F f) {
564   if (n) { each(n->l, f); f(n->val); each(n->r, f); }
752 }

e0e pair<Node*, Node*> split(Node* n, int k) {
b9e   if (!n) return {};
73c   if (cnt(n->l) >= k) { // "n->val >= k" for lower_bound(k)
fc5     auto [L,R] = split(n->l, k);
f96     n->l = R;
2f0     n->recalc();
```

```
fa5     return {L, n};
f71   } else {
3b6     auto {L,R} = split(n->r,k - cnt(n->l) - 1); // and just
      "k"

1b0     n->r = L;
2f0     n->recalc();
558     return {n, R};
639   }
e8b }
```

```
768 Node* merge(Node* l, Node* r) {
258   if (!l) return r;
584   if (!r) return l;
754   if (l->y > r->y) {
ed7     l->r = merge(l->r, r);
cc6     return l->recalc(), l;
f7c   } else {
654     r->l = merge(l, r->l);
a98     return r->recalc(), r;
520   }
159 }
```

```
139 Node* ins(Node* t, Node* n, int pos) {
286   auto {l,r} = split(t, pos);
e4f   return merge(merge(l, n), r);
32c }
```

```
// Example application: move the range [l, r) to index k
fr2 void move(Node*& t, int l, int r, int k) {
260   Node *a, *b, *c;
e14   tie(a,b) = split(t, l); tie(b,c) = split(b, r - 1);
386   if (k <= l) t = merge(ins(a, b, k), c);
236   else t = merge(a, ins(c, b, k - r));
9c4 }
```

FenwickTree.h

Description: Computes partial sums $a[0] + a[1] + \dots + a[pos - 1]$, and updates single elements $a[i]$, taking the difference between the old and new value.
Time: Both operations are $\mathcal{O}(\log N)$.

```
e62fac, 22 lines

066 struct FT {
cf7   vector<ll> s;
f03   FT(int n) : s(n) {}
cfe   void update(int pos, ll dif) { // a[pos] += dif
3e6     for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
a38   }
c6a   ll query(int pos) { // sum of values in [0, pos)
cd2     ll res = 0;
d2a     for (; pos > 0; pos &= pos - 1) res += s[pos-1];
b50     return res;
6de   }
6d8   int lower_bound(ll sum) { // min pos st sum of [0, pos] >=
      sum
      // Returns n if no sum is >= sum, or -1 if empty sum is

4b6   if (sum <= 0) return -1;
bec   int pos = 0;
888   for (int pw = 1 << 25; pw; pw >>= 1) {
4c6     if (pos + pw <= sz(s) && s[pos + pw-1] < sum)
7a3       pos += pw, sum -= s[pos-1];
63f   }
d75   return pos;
ea7   }
e62 };
```

FenwickTree2d.h

Description: Computes sums $a[i,j]$ for all $i < I, j < J$, and increases single elements $a[i,j]$. Requires that the elements to be updated are known in advance (call `fakeUpdate()` before `init()`).
Time: $\mathcal{O}(\log^2 N)$. (Use persistent segment trees for $\mathcal{O}(\log N)$.)

```
"FenwickTree.h"157f07, 22 lines

9a3 struct FT2 {
880   vector<vi> ys; vector<FT> ft;
6a4   FT2(int limx) : ys(limx) {}
5a4   void fakeUpdate(int x, int y) {
083     for (; x < sz(ys); x |= x + 1) ys[x].push_back(y);
01f   }
ca2   void init() {
a7a     for (vi& v : ys) sort(all(v)), ft.emplace_back(sz(v));
d5c   }
826   int ind(int x, int y) {
aee     return (int)(lower_bound(all(ys[x]), y) - ys[x].begin())
      ; }
eb5   void update(int x, int y, ll dif) {
a1f     for (; x < sz(ys); x |= x + 1)
593       ft[x].update(ind(x, y), dif);
bb1   }
cdc   ll query(int x, int y) {
5ff     ll sum = 0;
14f     for (; x; x &= x - 1)
99b       sum += ft[x-1].query(ind(x-1, y));
e66     return sum;
833   }
157 };
```

RMQ.h

Description: Range Minimum Queries on an array. Returns $\min(V[a], V[a + 1], \dots V[b - 1])$ in constant time.
Usage: `RMQ rmq(values);`
`rmq.query(inclusive, exclusive);`
Time: $\mathcal{O}(|V| \log |V| + Q)$

```
510c32, 16 lines

4fc template<class T>
76a struct RMQ {
b0a   vector<vector<T>> jmp;
38e   RMQ(const vector<T>& V) : jmp(1, V) {
a1b     for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
9d6       jmp.emplace_back(sz(V) - pw * 2 + 1);
939       rep(j,0,sz(jmp[k]))
d44         jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
288     }
e0a   }
0ad   T query(int a, int b) {
c7b     assert(a < b); // or return inf if a == b
e13     int dep = 31 - __builtin_clz(b - a);
7d3     return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
a3d   }
747 };
```

MoQueries.h

Description: Answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. If values are on tree edges, change step to add/remove the edge (a, c) and remove the initial add call (but keep in).
Time: $\mathcal{O}(N\sqrt{Q})$

```
a12ef4, 49 lines

ddb void add(int ind, int end) { ... } // add a[ind] (end = 0 or
1)
291 void del(int ind, int end) { ... } // remove a[ind]
5dd int calc() { ... } // compute current answer

aed vi mo(vector<pii> Q) {
903   int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
e06   vi s(sz(Q)), res = s;
```

```
a09 #define K(x) pii(x.first/blk, x.second ^ -(x.first/blk & 1))
0af   iota(all(s), 0);
c43   sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]);
      });
476   for (int qi : s) {
7f7     pii q = Q[qi];
a3d     while (L > q.first) add(--L, 0);
a58     while (R < q.second) add(R++, 1);
6b7     while (L < q.first) del(L++, 0);
e4a     while (R > q.second) del(--R, 1);
806     res[qi] = calc();
0f7   }
b50   return res;
e37 }
```

```
c35 vi moTree(vector<array<int, 2>> Q, vector<vi>& ed, int root
=0){
233   int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
ace   vi s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N);
74e   add(0, 0), in[0] = 1;
8e6   auto dfs = [&](int x, int p, int dep, auto& f) -> void {
a07     par[x] = p;
41b     L[x] = N;
2fe     if (dep) I[x] = N++;
86b     for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
340     if (!dep) I[x] = N++;
08a     R[x] = N;
329   };
219   dfs(root, -1, 0, dfs);
77f #define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk &
1))
0af   iota(all(s), 0);
c43   sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]);
      });
7b9   for (int qi : s) rep(end,0,2) {
ebe     int &a = pos[end], b = Q[qi][end], i = 0;
25d #define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
383                     else { add(c, end); in[c] = 1; } a = c; }
729     while (!(L[b] <= L[a] && R[a] <= R[b]))
efe       I[i++] = b, b = par[b];
dd2     while (a != b) step(par[a]);
82e     while (i-->) step(I[i]);
1fc     if (end) res[qi] = calc();
c88   }
b50   return res;
ce9 }
```

Numerical (4)

4.1 Polynomials and recurrences

Polynomial.h

```
c9b7b0, 17 lines

213 struct Poly {
3a1   vector<double> a;
9a5   double operator()(double x) const {
e3c     double val = 0;
d5c     for (int i = sz(a); i-->) (val *= x) += a[i];
d94     return val;
ae7   }
0ac   void diff() {
7b6     rep(i,1,sz(a)) a[i-1] = i*a[i];
468     a.pop_back();
afc   }
087   void divroot(double x0) {
898     double b = a.back(), c; a.back() = 0;
dbb     for(int i=sz(a)-1; i-->) c = a[i], a[i] = a[i+1]*x0+b, b
=c;
```

```
468     a.pop_back();
3f8   }
c9b   };
```

PolyRoots.h

Description: Finds the real roots to a polynomial.
Usage: polyRoots({{2,-3,1}},-1e9,1e9) // solve x²-3x+2 = 0
Time: $\mathcal{O}(n^2 \log(1/\epsilon))$

"Polynomial.h"	b00bfe, 23 lines
64a vector<double> polyRoots(Poly p, double xmin, double xmax) {	
853 if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }	
539 vector<double> ret;	
f55 Poly der = p;	
c06 der.diff();	
617 auto dr = polyRoots(der, xmin, xmax);	
d85 dr.push_back(xmin-1);	
12c dr.push_back(xmax+1);	
423 sort(all(dr));	
b98 rep(i,0,sz(dr)-1) {	
d85 double l = dr[i], h = dr[i+1];	
ad1 bool sign = p(l) > 0;	
b41 if (sign ^ (p(h) > 0)) {	
03d rep(it,0,60) { // while (h - l > 1e-8)	
761 double m = (l + h) / 2, f = p(m);	
0ac if ((f <= 0) ^ sign) l = m;	
193 else h = m;	
b69 }	
ff5 ret.push_back((l + h) / 2);	
fc2 }	
d15 }	
edf return ret;	
b00 }	

PolyInterpolate.h

Description: Given n points (x[i], y[i]), computes an n-1-degree polynomial *p* that passes through them: $p(x) = a[0] * x^0 + ... + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k / (n-1) * \pi), k = 0 \dots n-1$.
Time: $\mathcal{O}(n^2)$

	08bf48, 13 lines
ae0 typedef vector<double> vd;	
d97 vd interpolate(vd x, vd y, int n) {	
ce3 vd res(n), temp(n);	
155 rep(k,0,n-1) rep(i,k+1,n)	
e55 y[i] = (y[i] - y[k]) / (x[i] - x[k]);	
50e double last = 0; temp[0] = 1;	
bb6 rep(k,0,n) rep(i,0,n) {	
cc9 res[i] += y[k] * temp[i];	
c25 swap(last, temp[i]);	
fac temp[i] -= last * x[k];	
4c7 }	
b50 return res;	
285 }	

BerlekampMassey.h

Description: Recovers any *n*-order linear recurrence relation from the first *2n* terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$.
Usage: berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}
Time: $\mathcal{O}(N^2)$

"../number-theory/ModPow.h"	96548b, 20 lines
c10 vector<ll> berlekampMassey(vector<ll> s) {	
ea1 int n = sz(s), L = 0, m = 0;	
2a2 vector<ll> C(n), B(n), T;	
2b3 C[0] = B[0] = 1;	
d6f ll b = 1;	
36d rep(i,0,n) { ++m;	

b7f ll d = s[i] % mod;	
45a rep(j,1,L+1) d = (d + C[j] * s[i - j]) % mod;	
53a if (!d) continue;	
169 T = C; ll coef = d * modpow(b, mod-2) % mod;	
2d1 rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;	
b6c if (2 * L > i) continue;	
dc3 L = i + 1 - L; B = T; b = d; m = 0;	
8c2 }	
51b C.resize(L + 1); C.erase(C.begin());	
e98 for (ll& x : C) x = (mod - x) % mod;	
a91 return C;	
965 }	

LinearRecurrence.h

Description: Generates the *k*'th term of an *n*-order linear recurrence $S[i] = \sum_j S[i - j - 1]tr[j]$, given $S[0 \dots \geq n - 1]$ and $tr[0 \dots n - 1]$. Faster than matrix multiplication. Useful together with Berlekamp-Massey.
Usage: linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci number
Time: $\mathcal{O}(n^2 \log k)$

16c typedef vector<ll> Poly;	
2ef ll linearRec(Poly S, Poly tr, ll k) {	
327 int n = sz(tr);	
0e9 auto combine = [&](Poly a, Poly b) {	
b1c Poly res(n * 2 + 1);	
5f7 rep(i,0,n+1) rep(j,0,n+1)	
389 res[i + j] = (res[i + j] + a[i] * b[j]) % mod;	
bdc for (int i = 2 * n; i > n; --i) rep(j,0,n)	
fc3 res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) %	
mod;	
b76 res.resize(n + 1);	
b50 return res;	
55c };	
b18 Poly pol(n + 1), e(pol);	
997 pol[0] = e[1] = 1;	
e96 for (++k; k; k /= 2) {	
491 if (k % 2) pol = combine(pol, e);	
0d9 e = combine(e, e);	
813 }	
cd2 ll res = 0;	
e8d rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;	
b50 return res;	
594 }	

4.2 Optimization

GoldenSectionSearch.h

Description: Finds the argument minimizing the function *f* in the interval *[a,b]* assuming *f* is unimodal on the interval, i.e. has only one local minimum and no local maximum. The maximum error in the result is *eps*. Works equally well for maximization with a small change in the code. See Ternary-Search.h in the Various chapter for a discrete version.
Usage: double func(double x) { return 4+x+.3*x*x; }
double xmin = gss(-1000,1000,func);
Time: $\mathcal{O}(\log((b-a)/\epsilon))$

eb1 double gss(double a, double b, double (*f)(double)) {	
97e double r = (sqrt(5)-1)/2, eps = 1e-7;	
b87 double x1 = b - r*(b-a), x2 = a + r*(b-a);	
47d double f1 = f(x1), f2 = f(x2);	
708 while (b-a > eps)	
f4d if (f1 < f2) { //change to > to find maximum	
da5 b = x2; x2 = x1; f2 = f1;	
dfb x1 = b - r*(b-a); f1 = f(x1);	
451 } else {	

d6e a = x1; x1 = x2; f1 = f2;	
815 x2 = a + r*(b-a); f2 = f(x2);	
2fe }	
3f5 return a;	
31d }	

HillClimbing.h

Description: Poor man's optimization for unimodal functions

9eb typedef array<double, 2> P;	Seeef, 14 lines
0dd template<class F> pair<double, P> hillClimb(P start, F f) {	
b2f pair<double, P> cur(f(start), start);	
ba0 for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {	
6de rep(j,0,100) rep(dx,-1,2) rep(dy,-1,2) {	
0ce P p = cur.second;	
d9c p[0] += dx*jmp;	
e42 p[1] += dy*jmp;	
b52 cur = min(cur, make_pair(f(p), p));	
cc6 }	
8d9 }	
75e return cur;	
75c }	

Integrate.h

Description: Simple integration of a function over an interval using Simpson's rule. The error should be proportional to h^4 , although in practice you will want to verify that the result is stable to desired precision when epsilon changes.

044 template<class F>	
746 double quad(double a, double b, F f, const int n = 1000) {	
e8f double h = (b - a) / 2 / n, v = f(a) + f(b);	
ab6 rep(i,1,n*2)	
956 v += f(a + i*h) * (i&1 ? 4 : 2);	
19f return v * h / 3;	
ddc }	

IntegrateAdaptive.h

Description: Fast integration using an adaptive Simpson's rule.
Usage: double sphereVolume = quad(-1, 1, [](double x) {
return quad(-1, 1, [&](double y) {
return quad(-1, 1, [&](double z) {
return x*x + y*y + z*z < 1; });});});

070 typedef double d;	
1c8 #define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6	
044 template <class F>	
f19 d rec(F& f, d a, d b, d eps, d S) {	
1b4 d c = (a + b) / 2;	
1e8 d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;	
132 if (abs(T - S) <= 15 * eps b - a < 1e-10)	
f3d return T + (T - S) / 15;	
938 return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2,	
S2);	
720 }	
044 template<class F>	
dbb d quad(d a, d b, F f, d eps = 1e-8) {	
f1a return rec(f, a, b, eps, S(a, b));	
1e3 }	

Simplex.h

Description: Solves a general linear maximization problem: maximize $c^T x$ subject to $Ax \leq b, x \geq 0$. Returns -inf if there is no solution, inf if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. The input vector is set to an optimal *x* (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable.

Usage: vvd A = {{1,-1}, {-1,1}, {-1,-2}};
vd b = {1,1,-4}, c = {-1,-1}, x;
T val = LPSolver(A, b, c).solve(x);
Time: $\mathcal{O}(NM * \#pivots)$, where a pivot may be e.g. an edge relaxation.
 $\mathcal{O}(2^n)$ in the general case.

```
aa8530, 68 lines
943 typedef double T; // long double, Rational, double + modKp
    >...
487 typedef vector<T> vd;
840 typedef vector<vd> vvd;

8cb const T eps = 1e-8, inf = 1/.0;
85f #define MP make_pair
90c #define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s]))
    s=j

34b struct LPSolver {
b5c     int m, n;
14e     vi N, B;
d5f     vvd D;

9b8 LPSolver(const vvd& A, const vd& b, const vd& c) :
f40     m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
27d         rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
f03         rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] = b[i]
];}
59d         rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
24a         N[n] = -1; D[m+1][n] = 1;
6ff     }

333 void pivot(int r, int s) {
3cd     T *a = D[r].data(), inv = 1 / a[s];
12b     rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps) {
449         T *b = D[i].data(), inv2 = b[s] * inv;
e07         rep(j,0,n+2) b[j] -= a[j] * inv2;
e78         b[s] = a[s] * inv2;
ca4     }
485     rep(j,0,n+2) if (j != s) D[r][j] *= inv;
3b7     rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
dbd     D[r][s] = inv;
c97     swap(B[r], N[s]);
9cd }

24e bool simplex(int phase) {
8b8     int x = m + phase - 1;
1de     for (;;) {
7a6         int s = -1;
aea         rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
4dc         if (D[x][s] >= -eps) return true;
56d         int r = -1;
670         rep(i,0,m) {
77e             if (D[i][s] <= eps) continue;
c95             if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
133                 < MP(D[r][n+1] / D[r][s], B[r])) r = i;
468         }
fbf         if (r == -1) return false;
dba         pivot(r, s);
7d8     }
f15 }

859 T solve(vd &x) {
898     int r = 0;
435     rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
32d     if (D[r][n+1] < -eps) {
f65         pivot(r, n);
fda         if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
939         rep(i,0,m) if (B[i] == -1) {
37f             int s = 0;
c66             rep(j,1,n+1) ltj(D[i]);
```

```
b98     pivot(i, s);
683     }
b65     }
203     bool ok = simplex(1); x = vd(n);
972     rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
d3a     return ok ? D[m][n+1] : inf;
396     }
c57 };
```

4.3 Matrices

Determinant.h
Description: Calculates determinant of a matrix. Destroys the matrix.
Time: $\mathcal{O}(N^3)$

```
bd5cec, 15 lines
e36 double det(vector<vector<double>>& a) {
70e     int n = sz(a); double res = 1;
fea     rep(i,0,n) {
281         int b = i;
b0b         rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
311         if (i != b) swap(a[i], a[b]), res *= -1;
9b1         res *= a[i][i];
d5c         if (res == 0) return 0;
3e3         rep(j,i+1,n) {
f15             double v = a[j][i] / a[i][i];
353             if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
4ec         }
ee1     }
b50     return res;
bd5 }
```

IntDeterminant.h
Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version.
Time: $\mathcal{O}(N^3)$

```
3313dc, 18 lines
031 const ll mod = 12345;
38e ll det(vector<vector<ll>>& a) {
da9     int n = sz(a); ll ans = 1;
fea     rep(i,0,n) {
3e3         rep(j,i+1,n) {
f36             while (a[j][i] != 0) { // gcd step
479                 ll t = a[i][i] / a[j][i];
b87                 if (t) rep(k,i,n)
e5b                     a[i][k] = (a[i][k] - a[j][k] * t) % mod;
332                 swap(a[i], a[j]);
17c                 ans *= -1;
e81             }
30d         }
a97         ans = ans * a[i][i] % mod;
f4e         if (!ans) return 0;
f39     }
d38     return (ans + mod) % mod;
5e8 }
```

SolveLinear.h
Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost.
Time: $\mathcal{O}(n^2m)$

```
44c9ab, 38 lines
ae0 typedef vector<double> vd;
7bb const double eps = 1e-12;

40a int solveLinear(vector<vd>& A, vd& b, vd& x) {
6ed     int n = sz(A), m = sz(x), rank = 0, br, bc;
c74     if (n) assert(sz(A[0]) == m);
c38     vi col(m); iota(all(col), 0);

fea     rep(i,0,n) {
b44         double v, bv = 0;
```

```
b97     rep(r,i,n) rep(c,i,m)
684         if ((v = fabs(A[r][c])) > bv)
5fa             br = r, bc = c, bv = v;
2c9     if (bv <= eps) {
bc6         rep(j,i,n) if (fabs(b[j]) > eps) return -1;
c2b         break;
c92     }
bf5     swap(A[i], A[br]);
fd5     swap(b[i], b[br]);
f13     swap(col[i], col[bc]);
ede     rep(j,0,n) swap(A[j][i], A[j][bc]);
47f     bv = 1/A[i][i];
3e3     rep(j,i+1,n) {
d53         double fac = A[j][i] * bv;
2fd         b[j] -= fac * b[i];
924         rep(k,i+1,m) A[j][k] -= fac*A[i][k];
881     }
56e     rank++;
0f0 }

f8c x.assign(m, 0);
4b2 for (int i = rank; i--;) {
8b1     b[i] /= A[i][i];
563     x[col[i]] = b[i];
278     rep(j,0,i) b[j] -= A[j][i] * b[i];
ed1 }
cbb     return rank; // (multiple solutions if rank < m)
ade }
```

SolveLinear2.h
Description: To get all uniquely determined values of x back from SolveLinear, make the following changes:

```
08e495, 7 lines
" SolveLinear.h"
f94 rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
// ... then at the end:
9ff x.assign(m, undefined);
3ec rep(i,0,rank) {
76b     rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
ed6     x[col[i]] = b[i] / A[i][i];
878 fail;; }
```

SolveLinearBinary.h
Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b .
Time: $\mathcal{O}(n^2m)$

```
fa2d7a, 34 lines
983 typedef bitset<1000> bs;
```

```
6b7 int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
ab7     int n = sz(A), rank = 0, br;
758     assert(m <= sz(x));
c38     vi col(m); iota(all(col), 0);
fea     rep(i,0,n) {
3d2         for (br=i; br<n; ++br) if (A[br].any()) break;
099         if (br == n) {
52a             rep(j,i,n) if(b[j]) return -1;
c2b             break;
806         }
2d7         int bc = (int)A[br]._Find_next(i-1);
bf5         swap(A[i], A[br]);
fd5         swap(b[i], b[br]);
f13         swap(col[i], col[bc]);
bd2         rep(j,0,n) if (A[j][i] != A[j][bc]) {
e28             A[j].flip(i); A[j].flip(bc);
b44         }
8ec         rep(j,i+1,n) if (A[j][i]) {
41f             b[j] ^= b[i];
7dd             A[j] ^= A[i];
871         }
```

```
56e     rank++;
fe9   }

a08   x = bs();
4b2   for (int i = rank; i--;) {
c9a     if (!b[i]) continue;
57b     x[col[i]] = 1;
be1     rep(j,0,i) b[j] ^= A[j][i];
8fd   }
cbb   return rank; // (multiple solutions if rank < m)
26d }
```

MatrixInverse.h

Description: Invert matrix A . Returns rank; result is stored in A unless singular (rank < n). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of $A \bmod p$, and k is doubled in each step.
Time: $\mathcal{O}(n^3)$

ebfff6, 35 lines

```
4b5 int matInv(vector<vector<double>>& A) {
4d5     int n = sz(A); vi col(n);
b40     vector<vector<double>> tmp(n, vector<double>(n));
092     rep(i,0,n) tmp[i][i] = 1, col[i] = i;

fea     rep(i,0,n) {
903         int r = i, c = i;
786         rep(j,i,n) rep(k,i,n)
794             if (fabs(A[j][k]) > fabs(A[r][c]))
d4c                 r = j, c = k;
9a6         if (fabs(A[r][c]) < 1e-12) return i;
5d7         A[i].swap(A[r]); tmp[i].swap(tmp[r]);
433         rep(j,0,n)
90d             swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
c1d         swap(col[i], col[c]);
856         double v = A[i][i];
3e3         rep(j,i+1,n) {
d76             double f = A[j][i] / v;
9b5             A[j][i] = 0;
d60             rep(k,i+1,n) A[j][k] -= f*A[i][k];
6ac             rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
ebe         }
891         rep(j,i+1,n) A[i][j] /= v;
cce         rep(j,0,n) tmp[i][j] /= v;
e36         A[i][i] = 1;
26d     }

e95     for (int i = n-1; i > 0; --i) rep(j,0,i) {
44e         double v = A[j][i];
465         rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
03a     }

e26     rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
041     return n;
ebf }
```

MatrixInverse-mod.h

Description: Invert matrix A modulo a prime. Returns rank; result is stored in A unless singular (rank < n). For prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of $A \bmod p$, and k is doubled in each step.
Time: $\mathcal{O}(n^3)$

"../number-theory/ModPow.h"

0b7b13, 37 lines

```
702 int matInv(vector<vector<ll>>& A) {
4d5     int n = sz(A); vi col(n);
0ea     vector<vector<ll>> tmp(n, vector<ll>(n));
092     rep(i,0,n) tmp[i][i] = 1, col[i] = i;

fea     rep(i,0,n) {
```

```
903     int r = i, c = i;
b61     rep(j,i,n) rep(k,i,n) if (A[j][k]) {
d07         r = j; c = k; goto found;
b00     }
d9a     return i;
f5c found:
5d7     A[i].swap(A[r]); tmp[i].swap(tmp[r]);
433     rep(j,0,n)
90d         swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
c1d     swap(col[i], col[c]);
48e     ll v = modpow(A[i][i], mod - 2);
3e3     rep(j,i+1,n) {
b50         ll f = A[j][i] * v % mod;
9b5         A[j][i] = 0;
af9         rep(k,i+1,n) A[j][k] = (A[j][k] - f*A[i][k]) % mod;
9d6         rep(k,0,n) tmp[j][k] = (tmp[j][k] - f*tmp[i][k]) % mod;
;
3e0     }
73f     rep(j,i+1,n) A[i][j] = A[i][j] * v % mod;
f28     rep(j,0,n) tmp[i][j] = tmp[i][j] * v % mod;
e36     A[i][i] = 1;
41c }

e95     for (int i = n-1; i > 0; --i) rep(j,0,i) {
626         ll v = A[j][i];
f3a         rep(k,0,n) tmp[j][k] = (tmp[j][k] - v*tmp[i][k]) % mod;
b9a     }

605     rep(i,0,n) rep(j,0,n)
15c         A[col[i]][col[j]] = tmp[i][j] % mod + (tmp[i][j] < 0)*mod;
041     return n;
0b7 }
```

Tridiagonal.h

Description: $x = \text{tridiagonal}(d, p, q, b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i, 1 \leq i \leq n,$$

where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, \dots, -1, 1\}, \{0, c_1, c_2, \dots, c_n\}, \{b_1, b_2, \dots, b_n, 0\}, \{a_0, d_1, d_2, \dots, d_n, a_{n+1}\}).$$

Fails if the solution is not unique.
If $|d_i| > |p_i| + |q_{i-1}|$ for all i , or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither tr nor the check for diag[i] == 0 is needed.

8f9fa8, 26 lines

```
943 typedef double T;
1f8 vector<T> tridiagonal(vector<T> diag, const vector<T>& super
,
6d9     const vector<T>& sub, vector<T> b) {
a61     int n = sz(b); vi tr(n);
0a9     rep(i,0,n-1) {
2a3         if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] == 0
a5e             b[i+1] -= b[i] * diag[i+1] / super[i];
d7d             if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
eac             diag[i+1] = sub[i]; tr[++i] = 1;
464         } else {
```

```
8f9     diag[i+1] -= super[i]*sub[i]/diag[i];
361     b[i+1] -= b[i]*sub[i]/diag[i];
62d }
ed9 }
e3c for (int i = n; i--;) {
480     if (tr[i]) {
c4b         swap(b[i], b[i-1]);
eec         diag[i-1] = diag[i];
1b2         b[i] /= super[i-1];
054     } else {
b42         b[i] /= diag[i];
5d7         if (i) b[i-1] -= b[i]*super[i-1];
aa9     }
28a }
73f return b;
06d }
```

4.4 Fourier transforms

FastFourierTransform.h

Description: fft(a) computes $\hat{f}(k) = \sum_a a[x] \exp(2\pi i \cdot kx/N)$ for all k . N must be a power of 2. Useful for convolution: conv(a, b) = c, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n, reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16} ; higher for random inputs). Otherwise, use NTT/FFTMod.
Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ (~1s for $N = 2^{22}$)

00ced6, 35 lines

```
bcc typedef complex<double> C;
ae0 typedef vector<double> vd;
7c0 void fft(vector<C>& a) {
6ae     int n = sz(a), L = 31 - __builtin_clz(n);
f82     static vector<complex<long double>> R(2, 1);
991     static vector<C> rt(2, 1); // (^ 10% faster if double)
ad8     for (static int k = 2; k < n; k *= 2) {
e92         R.resize(n); rt.resize(n);
411         auto x = polar(1.0L, acos(-1.0L) / k);
c8b         rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
a8a     }
ebe     vi rev(n);
dc6     rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
47b     rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
657     for (int k = 1; k < n; k *= 2)
2cb         for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
e04             C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
20a             a[i + j + k] = a[i + j] - z;
1b0             a[i + j] += z;
577         }
01f }
28f vd conv(const vd& a, const vd& b) {
f88     if (a.empty() || b.empty()) return {};
698     vd res(sz(a) + sz(b) - 1);
441     int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
060     vector<C> in(n), out(n);
706     copy(all(a), begin(in));
fef     rep(i,0,sz(b)) in[i].imag(b[i]);
21a     fft(in);
6fb     for (C& x : in) x *= x;
4d7     rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
3d7     fft(out);
a15     rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
b50     return res;
873 }
```

FastFourierTransformMod.h

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$.

Time: $\mathcal{O}(N \log N)$, where $N = A + B $ (twice as slow as NTT or FFT)
"FastFourierTransform.h" b82773, 22 lines
<pre>192 typedef vector<ll> vl; 3fe template<int M> vl convMod(const vl &a, const vl &b) { f88 if (a.empty() b.empty()) return {}; 19d vl res(sz(a) + sz(b) - 1); a6f int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M)); 3dd vector<C> L(n), R(n), outs(n), outl(n); a1d rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut); 97d rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut); 5d5 fft(L), fft(R); fea rep(i,0,n) { 39d int j = -i & (n - 1); 65e outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n); 91a outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / li; cb3 } d08 fft(outl), fft(outs); 35e rep(i,0,sz(res)) { 351 ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5); 988 ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5); 6a3 res[i] = ((av % M * cut + bv) % M * cut + cv) % M; 58f } b50 return res; cif }</pre>

NumberTheoreticTransform.h

Description: ntt(a) computes $\hat{f}(k) = \sum_x a[x]g^{xk}$ for all k , where $g = \text{root}^{(mod-1)/N}$. N must be a power of 2. Useful for convolution modulo specific nice primes of the form $2^ab + 1$, where the convolution result has size at most 2^a . For arbitrary modulo, see FFTMod. conv(a, b) = c, where $c[x] = \sum a[i]b[x - i]$. For manual convolution: NTT the inputs, multiply pointwise, divide by n, reverse(start+1, end), NTT back. Inputs must be in $[0, \text{mod})$.

Time: $\mathcal{O}(N \log N)$

"../number-theory/ModPow.h" ced03d, 35 lines
<pre>b5e const ll mod = (119 << 23) + 1, root = 62; // = 998244353 // For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21 // and 483 << 21 (same root). The last two are > 10^9. 192 typedef vector<ll> vl; 8ec void ntt(vl &a) { 6ae int n = sz(a), L = 31 - __builtin_clz(n); 7c9 static vl rt(2, 1); 8ee for (static int k = 2, s = 2; k < n; k *= 2, s++) { 335 rt.resize(n); d43 ll z[] = {1, modpow(root, mod >> s)}; 8e7 rep(i,k,2*k) rt[i] = rt[i / 2] * z[i & 1] % mod; f39 } ebe vi rev(n); dcb rep(i,0,n) rev[i] = (rev[i / 2] (i & 1) << L) / 2; 47b rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]); 657 for (int k = 1; k < n; k *= 2) 2cb for (int i = 0; i < n; i += 2 * k) rep(j,0,k) { 86e ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j]; 598 a[i + j + k] = ai - z + (z > ai ? mod : 0); 589 ai += (ai + z >= mod ? z - mod : z); 9a8 } 3b7 } 08f vl conv(const vl &a, const vl &b) { f88 if (a.empty() b.empty()) return {}; f51 int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s), 570 n = 1 << B; 9ef int inv = modpow(n, mod - 2); e4c vl L(a), R(b), out(n); 6b4 L.resize(n), R.resize(n); d9e ntt(L), ntt(R); dfc rep(i,0,n) 0db out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod; ec9 ntt(out);</pre>

c20 return {out.begin(), out.begin() + s};
387 }

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x \oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.

Time: $\mathcal{O}(N \log N)$

464cf3, 16 lines
<pre>ac2 void FST(vi& a, bool inv) { a9d for (int n = sz(a), step = 1; step < n; step *= 2) { e24 for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step) { cde int &u = a[j], &v = a[j + step]; tie(u, v) = 078 inv ? pii(v - u, u) : pii(v, u + v); // AND 093 inv ? pii(v, u - v) : pii(u + v, u); // OR 79f pii(u + v, u - v); // XOR 7b7 } fae } 413 if (inv) for (int& x : a) x /= sz(a); // XOR only 57e } bd2 vi conv(vi a, vi b) { 595 FST(a, 0); FST(b, 0); bf9 rep(i,0,sz(a)) a[i] *= b[i]; 062 FST(a, 1); return a; 3cb }</pre>

Number theory (5)

5.1 Modular arithmetic

ModularArithmetic.h

Description: Operators for modular arithmetic. You need to set mod to some number first and then you can use the structure.

"euclid.h" ealac7, 19 lines
<pre>4eb const ll mod = 17; // change to something else bb6 struct Mod { 4ad ll x; 6cb Mod(ll y) : Mod(y%mod+mod,0){} 013 Mod(ll y,int) : x(y<mod?y:y-mod){} d79 Mod operator+(Mod b) { return {x + b.x,0}; } 66e Mod operator-(Mod b) { return {x - b.x + mod,0}; } 444 Mod operator*(Mod b) { return {x * b.x % mod,0}; } 882 Mod operator/(Mod b) { return *this * invert(b); } a58 Mod invert(Mod a) { 75a ll x, y, g = euclid(a.x, mod, x, y); e7e assert(g == 1); return x; 38b } 690 Mod operator^(ll e) { acd if (!e) return 1; a35 Mod r = *this ^ (e / 2); r = r * r; 825 return e&1 ? *this * r : r; a94 } c9c };</pre>

ModInverse.h

Description: Pre-computation of modular inverses. Assumes $\text{LIM} \leq \text{mod}$ and that mod is a prime.

6f684f, 3 lines
<pre>88a const ll mod = 1000000007, LIM = 200000; 66d ll* inv = new ll[LIM] - 1; inv[1] = 1; ab7 rep(i,2,LIM) inv[i] = mod - (mod / i) * inv[mod % i] % mod;</pre>

ModPow.h

b83e45, 8 lines
<pre>e2e const ll mod = 1000000007; // faster if const 9d8 ll modpow(ll b, ll e) {</pre>

d54 ll ans = 1;
36e for (; e; b = b * b % mod, e /= 2)
b46 if (e & 1) ans = ans * b % mod;
ba7 return ans;
d1e }

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. modLog(a,1,m) can be used to calculate the order of a .

Time: $\mathcal{O}(\sqrt{m})$

c040b8, 11 lines
<pre>0c8 ll modLog(ll a, ll b, ll m) { 462 ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1; 5cc unordered_map<ll, ll> A; 8b3 while (j <= n && (e = f = e * a % m) != b % m) ca6 A[e * b % m] = j++; 671 if (e == b % m) return j; fa7 if (__gcd(m, e) == __gcd(m, b)) 8ff rep(i,2,n+2) if (A.count(e = e * f % m)) da9 return n * i - A[e]; daa return -1; c04 }</pre>

ModSum.h

Description: Sums of mod'ed arithmetic progressions.

$\text{modsum}(\text{to}, c, k, m) = \sum_{i=0}^{\text{to}-1} (ki + c) \% m$. divsum is similar but for floored division.

Time: $\log(m)$, with a large constant.

5c5bc5, 16 lines
<pre>f4c typedef unsigned long long ull; a2d ull sumsq(ull to) { return to / 2 * ((to-1) 1); }</pre>
<pre>05c ull divsum(ull to, ull c, ull k, ull m) { d2f ull res = k / m * sumsq(to) + c / m * to; 98c k %= m; c %= m; 248 if (!k) return res; ff0 ull to2 = (to * k + c) / m; ea6 return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k); 78b }</pre>

<pre>fa0 ll modsum(ull to, ll c, ll k, ll m) { 6ef c = ((c % m) + m) % m; aa7 k = ((k % m) + m) % m; f6e return to * c + k * sumsq(to) - m * divsum(to, c, k, m); 5da }</pre>
--

ModMulLL.h

Description: Calculate $a \cdot b \pmod c$ (or $a^b \pmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.

Time: $\mathcal{O}(1)$ for modmul, $\mathcal{O}(\log b)$ for modpow

bbbb8f, 11 lines
<pre>f4c typedef unsigned long long ull; f85 ull modmul(ull a, ull b, ull M) { 2dd ll ret = a * b - M * ull(1.L / M * a * b); 964 return ret + M * (ret < 0) - M * (ret >= (ll)M); e93 } 4f6 ull modpow(ull b, ull e, ull mod) { c1a ull ans = 1; a18 for (; e; b = modmul(b, b, mod), e /= 2) 9e8 if (e & 1) ans = modmul(ans, b, mod); ba7 return ans; 100 }</pre>

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution).

Time: $\mathcal{O}(\log^2 p)$ worst case, $\mathcal{O}(\log p)$ for most p

19a793, 24 lines
<pre>"ModPow.h" a77 ll sqrt(ll a, ll p) { 5de a %= p; if (a < 0) a += p;</pre>

```
b47  if (a == 0) return 0;
5c6  assert(modpow(a, (p-1)/2, p) == 1); // else no solution
a75  if (p % 4 == 3) return modpow(a, (p+1)/4, p);
    // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
b94  ll s = p - 1, n = 2;
ee5  int r = 0, m;
084  while (s % 2 == 0)
082      ++r, s /= 2;
eaa  while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
0c3  ll x = modpow(a, (s + 1) / 2, p);
b74  ll b = modpow(a, s, p), g = modpow(n, s, p);
1af  for (;;) r = m) {
4fd      ll t = b;
713      for (m = 0; m < r && t != 1; ++m)
c58          t = t * t % p;
ae0      if (m == 0) return x;
20e      ll gs = modpow(g, 1LL << (r - m - 1), p);
fba      g = gs * gs % p;
4fb      x = x * gs % p;
c5c      b = b * g % p;
e3a  }
19a }
```

5.2 Primality

Eratosthenes.h

Description: Prime sieve for generating all primes up to a certain limit. isprime[i] is true iff i is a prime.
Time: lim=100'000'000 ≈ 0.8 s. Runs 30% faster if only odd indices are stored.

7c144c, 11 lines

```
129 const int MAX_PR = 5'000'000;
49e bitset<MAX_PR> isprime;
4e2 vi eratosthenesSieve(int lim) {
aae     isprime.set(); isprime[0] = isprime[1] = 0;
1f9     for (int i = 4; i < lim; i += 2) isprime[i] = 0;
b50     for (int i = 3; i*i < lim; i += 2) if (isprime[i])
f7e         for (int j = i*i; j < lim; j += i*2) isprime[j] = 0;
38c     vi pr;
920     rep(i,2,lim) if (isprime[i]) pr.push_back(i);
781     return pr;
d5c }
```

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than LIM.
Time: LIM=1e9 ≈ 1.5s

6b2912, 20 lines

```
2d0 const int LIM = 1e6;
a88 bitset<LIM> isPrime;
008 vi eratosthenes() {
2ff     const int S = (int)round(sqrt(LIM)), R = LIM / 2;
99c     vi pr = {2}, sieve(S+1); pr.reserve(int(LIM/log(LIM)*1.1))
    ;
6df     vector<pii> cp;
585     for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
c19         cp.push_back({i, i * i / 2});
c32         for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
d22     }
8b9     for (int L = 1; L <= R; L += S) {
efe         array<bool, S> block{};
069         for (auto &[p, idx] : cp)
84d             for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
a5a         rep(i,0,min(S, R - L))
b04             if (!block[i]) pr.push_back((L + i) * 2 + 1);
5b6     }
568     for (int i : pr) isPrime[i] = 1;
781     return pr;
8ee }
```

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
Time: 7 times the complexity of $a^b \bmod c$.

"ModMulLL.h"60dcd1, 12 lines

```
da4 bool isPrime(ull n) {
c16     if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
43a     ull A[] = {2, 325, 9375, 28178, 450775, 9780504,
1795265022},
c17         s = __builtin_ctzll(n-1), d = n >> s;
e80     for (ull a : A) { // ^ count trailing zeroes
6b4         ull p = modpow(a%n, d, n), i = s;
274         while (p != 1 && p != n - 1 && a % n && i--)
c77             p = modmul(p, p, n);
e28         if (p != n-1 && i != s) return 0;
edf     }
6a5     return 1;
60d }
```

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}(n^{1/4})$, less for numbers with small factors.

"ModMulLL.h", "MillerRabin.h"d8d98d, 18 lines

```
7eb ull pollard(ull n) {
222     ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
5f5     auto f = [&](ull x) { return modmul(x, x, n) + i; };
2ac     while (t++ % 40 || __gcd(prd, n) == 1) {
be9         if (x == y) x = ++i, y = f(x);
70f         if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
b78         x = f(x), y = f(f(y));
989     }
756     return __gcd(prd, n);
cd2 }
591 vector<ull> factor(ull n) {
1b9     if (n == 1) return {};
6b5     if (isPrime(n)) return {n};
bc6     ull x = pollard(n);
52a     auto l = factor(x), r = factor(n / x);
7af     l.insert(l.end(), all(r));
792     return l;
d54 }
```

5.3 Divisibility

euclid.h

Description: Finds two integers x and y, such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in __gcd instead. If a and b are coprime, then x is the inverse of a (mod b).

33ba8f, 5 lines

```
c22 ll euclid(ll a, ll b, ll &x, ll &y) {
1ee     if (!b) return x = 1, y = 0, a;
e3d     ll d = euclid(b, a % b, y, x);
0a4     return y -= a/b * x, d;
33b }
```

CRT.h

Description: Chinese Remainder Theorem.

crt(a, m, b, n) computes x such that $x \equiv a \pmod m$, $x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.
Time: log(n)

"euclid.h"04d93a, 7 lines

```
24a ll crt(ll a, ll m, ll b, ll n) {
08f     if (n > m) swap(a, b), swap(m, n);
58f     ll x, y, g = euclid(m, n, x, y);
522     assert((a - b) % g == 0); // else no solution
ef8     x = (b - a) % n * x % n / g * m + a;
```

```
16c     return x < 0 ? x + m*n/g : x;
04d }
```

5.3.1 Bézout's identity

For $a \neq, b \neq 0$, then $d = \gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

Description: Euler's φ function is defined as $\phi(n) := \#$ of positive integers $\leq n$ that are coprime with n. $\phi(1) = 1$, p prime $\Rightarrow \phi(p^k) = (p - 1)p^{k-1}$, m, n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1} p_2^{k_2} \dots p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1} \dots (p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n} (1 - 1/p)$.
 $\sum_{d|n} \phi(d) = n$, $\sum_{1 \leq k \leq n, \gcd(k, n) = 1} k = n\phi(n)/2, n > 1$

Euler's thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.

Fermat's little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod p \forall a$.

cf7d6d, 8 lines

```
fd7 const int LIM = 5000000;
c75 int phi[LIM];
```

```
d08 void calculatePhi() {
4d7     rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
c83     for (int i = 3; i < LIM; i += 2) if (phi[i] == i)
dc2         for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
043 }
```

5.4 Fractions

ContinuedFractions.h

Description: Given N and a real number $x \geq 0$, finds the closest rational approximation p/q with $p, q \leq N$. It will obey $|p/q - x| \leq 1/qN$.

For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. (p_k/q_k alternates between $> x$ and $< x$.) If x is rational, y eventually becomes ∞; if x is the root of a degree 2 polynomial the a's eventually become cyclic.

Time: $\mathcal{O}(\log N)$

dd6c5e, 21 lines

```
070 typedef double d; // for N ~ 1e7; long double for N ~ 1e9
c1b pair<ll, ll> approximate(d x, ll N) {
389     ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX; d y = x;
1de     for (;;) {
51e         ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ) / Q : inf)
```

```
    ,
57e         a = (ll)floor(y), b = min(a, lim),
c7d         NP = b*P + LP, NQ = b*Q + LQ;
f67     if (a > b) {
        // If b > a/2, we have a semi-convergent that gives
        // us a
        // better approximation; if b = a/2, we *may* have
        // one.
        // Return {P, Q} here for a more canonical
        // approximation.
735     return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q)) ?
```

```
    :
168         make_pair(NP, NQ) : make_pair(P, Q);
3ab     }
a7f     if (abs(y = 1/(y - (d)a)) > 3*N) {
d7d         return {NP, NQ};
f1d     }
6fe     LP = P; P = NP;
```

```
fe0      LQ = Q; Q = NQ;
543      }
ec1 }
```

FracBinarySearch.h

Description: Given f and N , finds the smallest fraction $p/q \in [0, 1]$ such that $f(p/q)$ is true, and $p, q \leq N$. You may want to throw an exception from f if it finds an exact solution, in which case N can be removed.
Usage: `fracBS([](Frac f) { return f.p>=3*f.q; }, 10);` // {1,3}
Time: $\mathcal{O}(\log(N))$

```
27ab3e, 25 lines
fea struct Frac { ll p, q; };

044 template<class F>
133 Frac fracBS(F f, ll N) {
028     bool dir = 1, A = 1, B = 1;
21b     Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
ccb     if (f(lo)) return lo;
d01     assert(f(hi));
053     while (A || B) {
955         ll adv = 0, step = 1; // move hi if dir, else lo
416         for (int si = 0; step; (step *= 2) >= si) {
964             adv += step;
296             Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
114             if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
7c8                 adv -= step; si = 2;
cac             }
d6d         }
adf         hi.p += lo.p * adv;
39e         hi.q += lo.q * adv;
9d2         dir = !dir;
4eb         swap(lo, hi);
441         A = B; B = !adv;
7df     }
7e3     return dir ? hi : lo;
ef9 }
```

5.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

5.6 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a , except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.7 Estimates

$$\sum_{d \mid n} d = O(n \log \log n).$$

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

5.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d \mid n} f(d) \Leftrightarrow f(n) = \sum_{d \mid n} \mu(d) g(n/d)$$

Other useful formulas/forms:

$$\sum_{d \mid n} \mu(d) = [n = 1] \text{ (very useful)}$$

$$g(n) = \sum_{n \mid d} f(d) \Leftrightarrow f(n) = \sum_{n \mid d} \mu(d/n) g(d)$$

$$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m) g(\lfloor \frac{n}{m} \rfloor)$$

Combinatorial (6)

6.1 Permutations

6.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

IntPerm.h

Description: Permutation -> integer conversion. (Not order preserving.) Integer -> permutation can use a lookup table.
Time: $\mathcal{O}(n)$

```
044568, 6 lines
aeb int permToInt(vi& v) {
fe8     int use = 0, i = 0, r = 0;
1d8     for(int x:v) r = r * ++i + __builtin_popcount(use & -(1<<x
    )),
7ca         use |= 1 << x; // (note: minus, not
    ~!)
4c1     return r;
044 }
```

6.1.2 Cycles

Let $g_S(n)$ be the number of n -permutations whose cycle lengths all belong to the set S . Then

$$\sum_{n=0}^\infty g_S(n) \frac{x^n}{n!} = \exp \left(\sum_{n \in S} \frac{x^n}{n} \right)$$

6.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rfloor$$

6.1.4 Burnside’s lemma

Given a group G of symmetries and a set X , the number of elements of X up to symmetry equals

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts “configurations” (of some sort) of length n , we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n, k)) = \frac{1}{n} \sum_{k \mid n} f(k) \phi(n/k).$$

6.2 Partitions and subsets

6.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1, \quad p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k - 1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	~2e5	~2e8

6.2.2 Lucas’ Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + \dots + n_1 p + n_0$ and $m = m_k p^k + \dots + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^k \binom{n_i}{m_i} \pmod{p}$.

6.2.3 Binomials

multinomial.h

Description: Computes $\binom{k_1 + \dots + k_n}{k_1, k_2, \dots, k_n} = \frac{(\sum k_i)!}{k_1! k_2! \dots k_n!}$.

```
a0a312, 5 lines
f7d ll multinomial(vi& v) {
015     ll c = 1, m = v.empty() ? 1 : v[0];
74f     rep(i, 1, sz(v)) rep(j, 0, v[i]) c = c * ++m / (j+1);
807     return c;
a0a }
```

6.3 General purpose numbers

6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t - 1}$ (FFT-able).
 $B[0, \dots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \dots]$

Sums of powers:

$$\sum_{i=1}^n n^m = \frac{1}{m+1} \sum_{k=0}^m \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^\infty f(i) = \int_m^\infty f(x) dx - \sum_{k=1}^\infty \frac{B_k}{k!} f^{(k-1)}(m)$$

$$\approx \int_m^\infty f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

6.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k), \; c(0,0) = 1$$
$$\sum_{k=0}^n c(n,k)x^k = x(x+1)\dots(x+n-1)$$

$$c(8,k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$$
$$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \dots$$

6.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j :s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j :s s.t. $\pi(j) \geq j$, k j :s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$
$$E(n,0) = E(n,n-1) = 1$$
$$E(n,k) = \sum_{j=0}^k (-1)^j \binom{n+1}{j} (k+1-j)^n$$

6.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$
$$S(n,1) = S(n,n) = 1$$
$$S(n,k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

6.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$. For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod{p}$$

6.3.6 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i : $n_1n_2\cdots n_kn^{k-2}$
with degrees d_i : $(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

6.3.7 Catalan numbers

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$
$$C_0 = 1, \; C_{n+1} = \frac{2(2n+1)}{n+2}C_n, \; C_{n+1} = \sum C_iC_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$

- sub-diagonal monotone paths in an $n \times n$ grid.

- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

Graph (7)

7.1 Fundamentals

BellmanFord.h

Description: Calculates shortest paths from s in a graph that might have negative edge weights. Unreachable nodes get dist = inf; nodes reachable through negative-weight cycles get dist = -inf. Assumes $V^2 \max |w_i| < \sim 2^{63}$.
Time: $\mathcal{O}(VE)$

```
830a8f, 23 lines
```

```
f5e const ll inf = LLONG_MAX;
83a struct Ed { int a, b, w, s() { return a < b ? a : -a; }};
9ac struct Node { ll dist = inf; int prev = -1; };

8d3 void bellmanFord(vector<Node>& nodes, vector<Ed>& eds, int s) {
    nodes[s].dist = 0;
    sort(all(eds), [](Ed a, Ed b) { return a.s() < b.s(); });

    int lim = sz(nodes) / 2 + 2; // /3+100 with shuffled vertices
    rep(i,0,lim) for (Ed ed : eds) {
        Node cur = nodes[ed.a], &dest = nodes[ed.b];
        if (abs(cur.dist) == inf) continue;
        ll d = cur.dist + ed.w;
        if (d < dest.dist) {
            dest.prev = ed.a;
            dest.dist = (i < lim-1 ? d : -inf);
        }
    }
    rep(i,0,lim) for (Ed e : eds) {
        if (nodes[e.a].dist == -inf)
            nodes[e.b].dist = -inf;
    }
}
```

FloydWarshall.h

Description: Calculates all-pairs shortest path in a directed graph that might have negative edge weights. Input is an distance matrix m , where $m[i][j] = \text{inf}$ if i and j are not adjacent. As output, $m[i][j]$ is set to the shortest distance between i and j , inf if no path, or -inf if the path goes through a negative-weight cycle.
Time: $\mathcal{O}(N^3)$

```
531245, 12 lines
```

```
964 const ll inf = 1LL << 62;
914 void floydWarshall(vector<vector<ll>>& m) {
    e9d int n = sz(m);
    831 rep(i,0,n) m[i][i] = min(m[i][i], 0LL);
    99d rep(k,0,n) rep(i,0,n) rep(j,0,n)
        19b if (m[i][k] != inf && m[k][j] != inf) {
            6e8 auto newDist = max(m[i][k] + m[k][j], -inf);
            e89 m[i][j] = min(m[i][j], newDist);
            f38 }
    a69 rep(k,0,n) if (m[k][k] < 0) rep(i,0,n) rep(j,0,n)
        ffd if (m[i][k] != inf && m[k][j] != inf) m[i][j] = -inf;
    f12 }
```

TopoSort.h

Description: Topological sorting. Given is an oriented graph. Output is an ordering of vertices, such that there are edges only from left to right. If there are cycles, the returned list will have size smaller than n – nodes reachable from cycles will not be returned.
Time: $\mathcal{O}(|V| + |E|)$

```
d678d8, 8 lines
```

```
01e vi topoSort(const vector<vi>& gr) {
654 vi indeg(sz(gr)), q;
618 for (auto& li : gr) for (int x : li) indeg[x]++;
ca7 rep(i,0,sz(gr)) if (indeg[i] == 0) q.push_back(i);
fd0 rep(j,0,sz(q)) for (int x : gr[q[j]])
bb2 if (--indeg[x] == 0) q.push_back(x);
bef return q;
d67 }
```

7.2 Network flow

PushRelabel.h

Description: Push-relabel using the highest label selection rule and the gap heuristic. Quite fast in practice. To obtain the actual flow, look at positive values only.
Time: $\mathcal{O}(V^2\sqrt{E})$

```
0ae1d4, 48 lines
```

```
49f struct PushRelabel {
e9b struct Edge {
548 int dest, back;
e00 ll f, c;
571 };
ed3 vector<vector<Edge>> g;
51c vector<ll> ec;
658 vector<Edge*> cur;
d9d vector<vi> hs; vi H;
4e1 PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {}

b1c void addEdge(int s, int t, ll cap, ll rcap=0) {
50b if (s == t) return;
    g[s].push_back({t, sz(g[t]), 0, cap});
2aa g[t].push_back({s, sz(g[s])-1, 0, rcap});
817 }

359 void addFlow(Edge& e, ll f) {
759 Edge &back = g[e.dest][e.back];
f7e if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
d2e e.f += f; e.c -= f; ec[e.dest] += f;
c47 back.f -= f; back.c += f; ec[back.dest] -= f;
340 }
0e0 ll calc(int s, int t) {
f00 int v = sz(g); H[s] = v; ec[t] = 1;
a6d vi co(2*v); co[0] = v-1;
fef rep(i,0,v) cur[i] = g[i].data();
8c2 for (Edge& e : g[s]) addFlow(e, e.c);

604 for (int hi = 0;;) {
ae9 while (hs[hi].empty()) if (!hi--) return -ec[s];
c6f int u = hs[hi].back(); hs[hi].pop_back();
a3e while (ec[u] > 0) // discharge u
457 if (cur[u] == g[u].data() + sz(g[u])) {
e94 H[u] = 1e9;
e19 for (Edge& e : g[u]) if (e.c && H[u] > H[e.dest]
] + 1)
    H[u] = H[e.dest] + 1, cur[u] = &e;
8c5 if (++co[H[u]], !--co[hi] && hi < v)
683 rep(i,0,v) if (hi < H[i] && H[i] < v)
    021 --co[H[i]], H[i] = v + 1;
3a2 hi = H[u];
aaf } else if (cur[u]->c && H[u] == H[cur[u]->dest] + 1)
779 addFlow(*cur[u], min(ec[u], cur[u]->c));
92c else ++cur[u];
120 }
```

12

```
29i    }
e84    bool leftOfMinCut(int a) { return H[a] >= sz(g); }
0ae    };
```

MinCostMaxFlow.h

Description: Min-cost max-flow. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only.
Time: $\mathcal{O}(FE \log(V))$ where F is max flow. $\mathcal{O}(VE)$ for setpi.

```
58385b, 79 lines
c4d #include <bits/extc++.h>

9f4 const ll INF = numeric_limits<ll>::max() / 4;

6f3 struct MCMF {
670     struct edge {
ede         int from, to, rev;
e20         ll cap, cost, flow;
092     };
060     int N;
091     vector<vector<edge>> ed;
605     vi seen;
0ec     vector<ll> dist, pi;
c45     vector<edge*> par;

f01     MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}

6f3     void addEdge(int from, int to, ll cap, ll cost) {
fda         if (from == to) return;
1af         ed[from].push_back(edge{ from,to,sz(ed[to]),cap,cost,0 }
);
700         ed[to].push_back(edge{ to,from,sz(ed[from])-1,0,-cost,0
});
c71     }

975     void path(int s) {
7d4         fill(all(seen), 0);
04e         fill(all(dist), INF);
799         dist[s] = 0; ll di;

937     __gnu_pbds::priority_queue<pair<ll, int>> q;
9fb     vector<decltype(q)::point_iterator> its(N);
23b     q.push({ 0, s });

14d     while (!q.empty()) {
eda         s = q.top().second; q.pop();
2af         seen[s] = 1; di = dist[s] + pi[s];
edd         for (edge& e : ed[s]) if (!seen[e.to]) {
f1f             ll val = di - pi[e.to] + e.cost;
f3c             if (e.cap - e.flow > 0 && val < dist[e.to]) {
0c7                 dist[e.to] = val;
fb6                 par[e.to] = &e;
b17                 if (its[e.to] == q.end())
aac                     its[e.to] = q.push({ -dist[e.to], e.to });
295                 else
36c                     q.modify(its[e.to], { -dist[e.to], e.to });
ca0             }
4cd         }
062     }
e21     rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
7e4 }

310 pair<ll, ll> maxflow(int s, int t) {
3d3     ll totflow = 0, totcost = 0;
8dd     while (path(s), seen[t]) {
535         ll fl = INF;
1de         for (edge* x = par[t]; x; x = par[x->from])
8ed             fl = min(fl, x->cap - x->flow);
```

```
f9f         totflow += fl;
733         for (edge* x = par[t]; x; x = par[x->from]) {
10b             x->flow += fl;
e58             ed[x->to][x->rev].flow -= fl;
3bf         }
8d9     }
67b     rep(i,0,N) for(edge& e : ed[i]) totcost += e.cost * e.
flow;
17e     return {totflow, totcost/2};
24f }

// If some costs can be negative, call this before
maxflow:
e69 void setpi(int s) { // (otherwise, leave this out)
a3d     fill(all(pi), INF); pi[s] = 0;
a10     int it = N, ch = 1; ll v;
036     while (ch-- && it--)
edb         rep(i,0,N) if (pi[i] != INF)
fbo             for (edge& e : ed[i]) if (e.cap)
257                 if ((v = pi[i] + e.cost) < pi[e.to])
a43                     pi[e.to] = v, ch = 1;
38b         assert(it >= 0); // negative cost cycle
684     }
b36 };
```

EdmondsKarp.h

Description: Flow algorithm with guaranteed complexity $\mathcal{O}(VE^2)$. To get edge flow values, compare capacities before and after, and take the positive values only.

```
482fe0, 36 lines
0e6 template<class T> T edmondsKarp(vector<unordered_map<int, T
>>&
327     graph, int source, int sink) {
65f     assert(source != sink);
8f6     T flow = 0;
1d5     vi par(sz(graph)), q = par;

1de     for (;;) {
ee9         fill(all(par), -1);
73a         par[source] = 0;
e97         int ptr = 1;
fae         q[0] = source;

772         rep(i,0,ptr) {
912             int x = q[i];
31c             for (auto e : graph[x]) {
299                 if (par[e.first] == -1 && e.second > 0) {
d95                     par[e.first] = x;
e70                     q[ptr++] = e.first;
f64                     if (e.first == sink) goto out;
3a4                 }
6e8             }
56e         }
99d         return flow;
3a3     out:
c55     T inc = numeric_limits<T>::max();
bd6     for (int y = sink; y != source; y = par[y])
7d1         inc = min(inc, graph[par[y]][y]);

9c6     flow += inc;
65c     for (int y = sink; y != source; y = par[y]) {
792         int p = par[y];
f35         if ((graph[p][y] -= inc) <= 0) graph[p].erase(y);
4ee         graph[y][p] += inc;
548     }
ff8 }
261 }
```

Dinic.h

Description: Flow algorithm with complexity $\mathcal{O}(VE \log U)$ where $U = \max|cap|$. $\mathcal{O}(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $\mathcal{O}(\sqrt{VE})$ for bipartite matching.

```
d7f0f1, 42 lines
14d struct Dinic {
e9b     struct Edge {
c63         int to, rev;
78d         ll c, oc;
181         ll flow() { return max(oc - c, 0LL); } // if you need
flows
d46     };
2c7     vi lvl, ptr, q;
c9e     vector<vector<Edge>> adj;
c40     Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
31e     void addEdge(int a, int b, ll c, ll rcap = 0) {
117         adj[a].push_back({b, sz(adj[b]), c, c});
0fc         adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
01a     }
136     ll dfs(int v, int t, ll f) {
323         if (v == t || !f) return f;
00f         for (int& i = ptr[v]; i < sz(adj[v]); i++) {
e3c             Edge& e = adj[v][i];
e24             if (lvl[e.to] == lvl[v] + 1)
d0c                 if (ll p = dfs(e.to, t, min(f, e.c))) {
06b                     e.c -= p, adj[e.to][e.rev].c += p;
74e                     return p;
494                 }
f73             }
bb3             return 0;
092     }
0e0     ll calc(int s, int t) {
2c5         ll flow = 0; q[0] = s;
d59         rep(L,0,31) do { // 'int L=30' maybe faster for random
data
fec             lvl = ptr = vi(sz(q));
d46             int qi = 0, qe = lvl[s] = 1;
aa5             while (qi < qe && !lvl[t]) {
7da                 int v = q[qi++];
d2f                 for (Edge e : adj[v])
4da                     if (!lvl[e.to] && e.c >> (30 - L))
12a                         q[qi++] = e.to, lvl[e.to] = lvl[v] + 1;
80e             }
0f3             while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
964             while (lvl[t]);
99d             return flow;
3eb         }
74b         bool leftOfMinCut(int a) { return lvl[a] != 0; }
d7f     };
```

MinCut.h

Description: After running max-flow, the left side of a min-cut from s to t is given by all vertices reachable from s , only traversing edges with positive residual capacity.

GlobalMinCut.h

Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.

```
8b0e19, 21 lines
192 pair<int, vi> globalMinCut(vector<vi> mat) {
afa     pair<int, vi> best = {INT_MAX, {}};
755     int n = sz(mat);
91d     vector<vi> co(n);
d0f     rep(i,0,n) co[i] = {i};
488     rep(ph,1,n) {
2e9         vi w = mat[0];
e44         size_t s = 0, t = 0;
```

```
694 rep(it,0,n-ph) { // O(V^2) -> O(E log V) with prio.
queue
d6e w[t] = INT_MIN;
a5f s = t, t = max_element(all(w)) - w.begin();
d39 rep(i,0,n) w[i] += mat[t][i];
ec9 }
3df best = min(best, {w[t] - mat[t][t], co[t]});
096 co[s].insert(co[s].end(), all(co[t]));
959 rep(i,0,n) mat[s][i] += mat[t][i];
984 rep(i,0,n) mat[i][s] = mat[s][i];
5dd mat[0][t] = INT_MIN;
ca0 }
f26 return best;
8b0 }
```

GomoryHu.h
Description: Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.
Time: $\mathcal{O}(V)$ Flow Computations

"PushRelabel.h"	0418b3, 13 lines
-----------------	------------------

```
2d0 typedef array<ll, 3> Edge;
71a vector<Edge> gomoryHu(int N, vector<Edge> ed) {
92a vector<Edge> tree;
025 vi par(N);
e64 rep(i,1,N) {
30d PushRelabel D(N); // Dinic also works
24b for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
337 tree.push_back({i, par[i], D.calc(i, par[i])});
c70 rep(j,i+1,N)
b7b if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
93c }
9cb return tree;
65c }
```

7.3 Matching

HopcroftKarp.h
Description: Fast bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and r should be a vector full of -1 's of the same size as the right partition. Returns the size of the matching. $r[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Time: $\mathcal{O}(E\sqrt{V})$

	731cfb, 20 lines
--	------------------

```
674 int hopcroftKarp(vector<vi>& g, vi& r) {
bfd int n = sz(g), res = 0;
b55 vi l(n, -1), q(n), d(n);
4a3 auto dfs = [&](auto f, int u) -> bool {
513 int t = exchange(d[u], 0) + 1;
4fb for (int v : g[u])
1a2 if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
106 return l[u] = v, r[v] = u, 1;
bb3 return 0;
a95 };
a44 for (int t = 0, f = 0; t = f = 0, d.assign(n, 0)) {
96b rep(i,0,n) if (l[i] == -1) q[t++] = i, d[i] = 1;
57e rep(i,0,t) for (int v : g[q[i]]) {
15f if (r[v] == -1) f = 1;
fa5 else if (!d[r[v]]) d[r[v]] = d[q[i]] + 1, q[t++] = r[v];
64a }
f02 if (!f) return res;
c97 rep(i,0,n) if (l[i] == -1) res += dfs(dfs, i);
cdf }
731 }
```

DFSMatching.h
Description: Simple bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1 's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Usage: vi $btoa(m, -1)$; $dfsMatching(g, btoa)$;
Time: $\mathcal{O}(VE)$

	522b98, 22 lines
--	------------------

```
a47 bool find(int j, vector<vi>& g, vi& btoa, vi& vis) {
107 if (btoa[j] == -1) return 1;
d11 vis[j] = 1; int di = btoa[j];
c00 for (int e : g[di])
800 if (!vis[e] && find(e, g, btoa, vis)) {
a5e btoa[e] = di;
6a5 return 1;
6ba }
bb3 return 0;
d13 }
0d5 int dfsMatching(vector<vi>& g, vi& btoa) {
dd3 vi vis;
a7f rep(i,0,sz(g)) {
d3b vis.assign(sz(btoa), 0);
43f for (int j : g[i])
4df if (find(j, g, btoa, vis)) {
e4b btoa[j] = i;
c2b break;
829 }
df2 }
00d return sz(btoa) - (int)count(all(btoa), -1);
f24 }
```

MinimumVertexCover.h
Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.

"DFSMatching.h"	da4196, 20 lines
-----------------	------------------

```
0ba vi cover(vector<vi>& g, int n, int m) {
044 vi match(m, -1);
92b int res = dfsMatching(g, match);
80e vector<bool> lfound(n, true), seen(m);
a5b for (int it : match) if (it != -1) lfound[it] = false;
baf vi q, cover;
2f1 rep(i,0,n) if (lfound[i]) q.push_back(i);
14d while (!q.empty()) {
5c3 int i = q.back(); q.pop_back();
f89 lfound[i] = 1;
e86 for (int e : g[i]) if (!seen[e] && match[e] != -1) {
430 seen[e] = true;
4c1 q.push_back(match[e]);
46e }
069 }
64c rep(i,0,n) if (!lfound[i]) cover.push_back(i);
f81 rep(i,0,m) if (seen[i]) cover.push_back(n+i);
b59 assert(sz(cover) == res);
64f return cover;
da4 }
```

WeightedMatching.h
Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes $cost[N][M]$, where $cost[i][j]$ = cost for $L[i]$ to be matched with $R[j]$, and returns $(min\ cost, match)$, where $L[i]$ is matched with $R[match[i]]$. Negate costs for max cost. Requires $N \leq M$.
Time: $\mathcal{O}(N^2M)$

	8f41c7, 24 lines
--	------------------

```
4fc template<class T>
1f4 pair<T, vi> weightedMatching(vector<vector<T>>& C) {
1fb int i = sz(C), m = i ? sz(C[0]) : 0, c, s, r;
f5e vector<T> dist(m), pot(m);
```

```
002 vi match(i), rev(m, -1), cols(m), prev(m);
000 T d = 0, nd, cost = 0;
b12 while (i--) {
388 rep(c,0,m) dist[c] = C[i][c], cols[c] = c, prev[c] = i;
344 for (s = 0;;) {
5da rep(j,s,m) {
56b c = cols[j], nd = dist[c] - pot[c];
445 if (j == s || d > nd) d = nd, swap(cols[s], cols[j]);
;
c6d }
785 if ((r = rev[c = cols[s++]]) == -1) break;
929 rep(j,0,m) if (dist[j] > (nd = C[r][j]-C[r][c]+dist[c]))
dist[j] = nd, prev[j] = r;
d1e }
da4 cost += dist[c];
827 while (s-->0) pot[cols[s]] = dist[cols[s]] - d;
2fc for (; r != i; swap(c, match[r]))
7f3 r = rev[c] = prev[c];
6f6 }
239 return {cost, match};
f04 }
```

GeneralMatching.h
Description: Matching for general graphs. Fails with probability N/mod .
Time: $\mathcal{O}(N^3)$

"../numerical/MatrixInverse-mod.h"	cb1912, 40 lines
------------------------------------	------------------

```
376 vector<pii> generalMatching(int N, vector<pii>& ed) {
30b vector<vector<ll>> mat(N, vector<ll>(N)), A;
c3f for (pii pa : ed) {
436 int a = pa.first, b = pa.second, r = rand() % mod;
7c6 mat[a][b] = r, mat[b][a] = (mod - r) % mod;
1c6 }

6b0 int r = matInv(A = mat), M = 2*N - r, fi, fj;
5a6 assert(r % 2 == 0);

076 if (M != N) do {
139 mat.resize(M, vector<ll>(M));
5c8 rep(i,0,N) {
4c4 mat[i].resize(M);
8fa rep(j,N,M) {
eb1 int r = rand() % mod;
b8a mat[i][j] = r, mat[j][i] = (mod - r) % mod;
96e }
7e9 }
e97 } while (matInv(A = mat) != M);

cd6 vi has(M, 1); vector<pii> ret;
4ee rep(it,0,M/2) {
a76 rep(i,0,M) if (has[i])
539 rep(j,i+1,M) if (A[i][j] && mat[i][j]) {
d5a fi = i; fj = j; goto done;
d9f } assert(0); done:
8f3 if (fj < N) ret.emplace_back(fi, fj);
5b1 has[fi] = has[fj] = 0;
99f rep(sw,0,2) {
6e1 ll a = modpow(A[fi][fj], mod-2);
1ed rep(i,0,M) if (has[i] && A[i][fj]) {
2fb ll b = A[i][fj] * a % mod;
a44 rep(j,0,M) A[i][j] = (A[i][j] - A[fi][j] * b) % mod;
79b }
80f swap(fi,fj);
a64 }
6e0 }
edf return ret;
cbl }
```

7.4 DFS algorithms

SCC.h

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa.

Usage: scc(graph, [&](vi& v) { ... }) visits all components in reverse topological order. comp[i] holds the component index of a node (a component only has edges to components with lower index). ncomps will contain the number of components.

Time: $\mathcal{O}(E + V)$

76b5c9, 24 lines	
508	vi val, comp, z, cont;
f12	int Time, ncomps;
61a	template<class G, class F> int dfs(int j, G& g, F& f) {
e35	int low = val[j] = ++Time, x; z.push_back(j);
c9e	for (auto e : g[j]) if (comp[e] < 0)
7ae	low = min(low, val[e] ?: dfs(e,g,f));
f5c	if (low == val[j]) {
016	do {
04f	x = z.back(); z.pop_back();
77d	comp[x] = ncomps;
c76	cont.push_back(x);
ae8	} while (x != j);
165	f(cont); cont.clear();
9df	ncomps++;
64c	}
c34	return val[j] = low;
351	}
56f	template<class G, class F> void scc(G& g, F f) {
625	int n = sz(g);
cd4	val.assign(n, 0); comp.assign(n, -1);
a71	Time = ncomps = 0;
561	rep(i,0,n) if (comp[i] < 0) dfs(i, g, f);
56b	}

BiconnectedComponents.h

Description: Finds all biconnected components in an undirected graph, and runs a callback for the edges in each. In a biconnected component there are at least two internally disjoint paths between any two nodes (a cycle exists through them). Note that a node can be in several components. An edge which is not in a component is a bridge, i.e., not part of any cycle.

Usage: int eid = 0; ed.resize(N);

for each edge (a,b) {
ed[a].emplace_back(b, eid);
ed[b].emplace_back(a, eid++); }
bicomps([&](const vi& edgelist) {...});

Time: $\mathcal{O}(E + V)$

c6b7c7, 32 lines	
16a	vi num, st;
f04	vector<vector<pii>> ed;
8b7	int Time;
044	template<class F>
cc8	int dfs(int at, int par, F& f) {
2fe	int me = num[at] = ++Time, top = me;
626	for (auto [y, e] : ed[at]) if (e != par) {
9ad	if (num[y]) {
61e	top = min(top, num[y]);
dbb	if (num[y] < me)
b3b	st.push_back(e);
1c2	} else {
d40	int si = sz(st);
828	int up = dfs(y, e, f);
f00	top = min(top, up);
5ce	if (up == me) {
b3b	st.push_back(e);
f24	f(vi(st.begin() + si, st.end()));
901	st.resize(si);
c92	}

984	else if (up < me) st.push_back(e);
2e7	else { /* e is a bridge */ }
739	}
bd7	}
3a6	return top;
59b	}
044	template<class F>
f1c	void bicomps(F f) {
fe6	num.assign(sz(ed), 0);
bd2	rep(i,0,sz(ed)) if (!num[i]) dfs(i, -1, f);
c03	}

2sat.h

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $(a||b)\&\&(!a||c)\&\&(d||b)\&\&...$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).

Usage: TwoSat ts(number of boolean variables);
ts.either(0, ~3); // Var 0 is true or var 3 is false
ts.setValue(2); // Var 2 is true
ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true
ts.solve(); // Returns true iff it is solvable
ts.values[0..N-1] holds the assigned values to the vars

Time: $\mathcal{O}(N + E)$, where N is the number of boolean variables, and E is the number of clauses.

5f9706, 56 lines	
d9d	struct TwoSat {
060	int N;
330	vector<vi> gr;
0f0	vi values; // 0 = false, 1 = true
f9e	TwoSat(int n = 0) : N(n), gr(2*n) {}
7ff	int addVar() { // (optional)
56e	gr.emplace_back();
56e	gr.emplace_back();
d17	return N++;
7b5	}
363	void either(int f, int j) {
281	f = max(2*f, -1-2*f);
036	j = max(2*j, -1-2*j);
732	gr[f].push_back(j^1);
150	gr[j].push_back(f^1);
516	}
926	void setValue(int x) { either(x, x); }
827	void atMostOne(const vi& li) { // (optional)
615	if (sz(li) <= 1) return;
da9	int cur = ~li[0];
9d4	rep(i,2,sz(li)) {
b70	int next = addVar();
909	either(cur, ~li[i]);
86e	either(cur, next);
d1a	either(~li[i], next);
072	cur = ~next;
8d3	}
ed7	either(cur, ~li[1]);
10f	}
7b1	vi val, comp, z; int time = 0;
a84	int dfs(int i) {
572	int low = val[i] = ++time, x; z.push_back(i);
3a3	for(int e : gr[i]) if (!comp[e])
1c8	low = min(low, val[e] ?: dfs(e));
339	if (low == val[i]) do {
04f	x = z.back(); z.pop_back();
7cc	comp[x] = low;

142	if (values[x>>1] == -1)
378	values[x>>1] = x&1;
b15	} while (x != i);
3e1	return val[i] = low;
ef5	}
6c0	bool solve() {
977	values.assign(N, -1);
6a0	val.assign(2*N, 0); comp = val;
e4c	rep(i,0,2*N) if (!comp[i]) dfs(i);
e72	rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0;
6a5	return 1;
2bb	}
5f9	};

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.

Time: $\mathcal{O}(V + E)$

780b64, 15 lines	
87a	vi eulerWalk(vector<vector<pii>>& gr, int nedges, int src=0)
	{
f0e	int n = sz(gr);
56e	vi D(n), its(n), eu(nedges), ret, s = {src};
9e7	D[src]++; // to allow Euler paths, not just cycles
980	while (!s.empty()) {
115	int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
041	if (it == end){ ret.push_back(x); s.pop_back(); continue
	; }
488	tie(y, e) = gr[x][it++];
8e6	if (!eu[e]) {
a02	D[x]--, D[y]++;
026	eu[e] = 1; s.push_back(y);
94d	}}
0e2	for (int x : D) if (x < 0 sz(ret) != nedges+1) return {
	};
6a7	return {ret.rbegin(), ret.rend()};
780	}

7.5 Coloring

EdgeColoring.h

Description: Given a simple, undirected graph with max degree D , computes a $(D + 1)$ -coloring of the edges such that no neighboring edges share a color. (D -coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)

Time: $\mathcal{O}(NM)$

e210c2, 31 lines	
f41	vi edgeColoring(int N, vector<pii> eds) {
727	vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
10d	for (pii e : eds) ++cc[e.first], ++cc[e.second];
e2f	int u, v, ncols = *max_element(all(cc)) + 1;
fda	vector<vi> adj(N, vi(ncols, -1));
6ec	for (pii e : eds) {
119	tie(u, v) = e;
e51	fan[0] = v;
0f4	loc.assign(ncols, 0);
696	int at = u, end = u, d, c = free[u], ind = 0, i = 0;
3b2	while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
3e1	loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
01e	cc[loc[d]] = c;
997	for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd
	])
4ff	swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
79f	while (adj[fan[i]][d] != -1) {
a9f	int left = fan[i], right = fan[++i], e = cc[i];
99b	adj[u][e] = left;

```
ccb      adj[left][e] = u;
f7e      adj[right][e] = -1;
d99      free[right] = e;
316     }
dfd      adj[u][d] = fan[i];
c45      adj[fan[i]][d] = u;
0e1      for (int y : {fan[0], u, end})
3fa          for (int& z = free[y] = 0; adj[y][z] != -1; z++);
fdc     }
29d     rep(i,0,sz(eds))
961         for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];
edf     return ret;
e21 }
```

7.6 Heuristics

MaximalCliques.h

Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.

Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs

b0d5b1, 12 lines

```
753 typedef bitset<128> B;
044 template<class F>
6a9 void cliques(vector<B>& eds, F f, B P = ~B(), B X={}, B R={})
    {
9bb     if (!P.any()) { if (!X.any()) f(R); return; }
a8e     auto q = (P | X)._Find_first();
cd1     auto cands = P & ~eds[q];
3d7     rep(i,0,sz(eds)) if (cands[i]) {
a75         R[i] = 1;
e78         cliques(eds, f, P & eds[i], X & eds[i], R);
bb6         R[i] = P[i] = 0; X[i] = 1;
181     }
c9d }
```

MaximumClique.h

Description: Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.

Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs faster for sparse graphs.

f7c0bc, 49 lines

```
54e typedef vector<bitset<200>> vb;
c7d struct Maxclique {
24e     double limit=0.025, pk=0;
c04     struct Vertex { int i, d=0; };
94c     typedef vector<Vertex> vv;
d44     vb e;
df7     vv V;
f97     vector<vi> C;
efb     vi qmax, q, S, old;
fe3     void init(vv& r) {
fd3         for (auto& v : r) v.d = 0;
583         for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
0f1         sort(all(r), [](auto a, auto b) { return a.d > b.d; });
c43         int mxD = r[0].d;
825         rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
7c4     }
bc8     void expand(vv& R, int lev = 1) {
ac1         S[lev] += S[lev - 1] - old[lev];
92c         old[lev] = S[lev - 1];
d18         while (sz(R)) {
3fd             if (sz(q) + R.back().d <= sz(qmax)) return;
d62             q.push_back(R.back().i);
f28             vv T;
96e             for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i
});
d21         if (sz(T)) {
```

```
eea     if (S[lev]++ / ++pk < limit) init(T);
457     int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1,
1);
9bc     C[1].clear(), C[2].clear();
969     for (auto v : T) {
bfe         int k = 1;
8f5         auto f = [&](int i) { return e[v.i][i]; };
5c6         while (any_of(all(C[k]), f)) k++;
782         if (k > mxk) mxk = k, C[mxk + 1].clear();
18a         if (k < mnk) T[j++] .i = v.i;
0e6         C[k].push_back(v.i);
322     }
238     if (j > 0) T[j - 1].d = 0;
75f     rep(k,mnk,mxk + 1) for (int i : C[k])
361         T[j].i = i, T[j++].d = k;
22d     expand(T, lev + 1);
2a0     } else if (sz(q) > sz(qmax)) qmax = q;
c81     q.pop_back(), R.pop_back();
876     }
f0a     }
1eb     vi maxClique() { init(V), expand(V); return qmax; }
b40     Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S)
    {
cd9         rep(i,0,sz(e)) V.push_back({i});
36a     }
b63 };
```

MaximumIndependentSet.h

Description: To obtain a maximum independent set of a graph, find a max clique of the complement. If the graph is bipartite, see MinimumVertex-Cover.

7.7 Trees

BinaryLifting.h

Description: Calculate power of two jumps in a tree, to support fast upward jumps and LCAs. Assumes the root node points to itself.

Time: construction $\mathcal{O}(N \log N)$, queries $\mathcal{O}(\log N)$

bfce85, 25 lines

```
750 vector<vi> treeJump(vi& P){
312     int on = 1, d = 1;
d0a     while(on < sz(P)) on *= 2, d++;
5ef     vector<vi> jmp(d, P);
959     rep(i,1,d) rep(j,0,sz(P))
884         jmp[i][j] = jmp[i-1][jmp[i-1][j]];
ba4     return jmp;
6d3 }

1e7 int jmp(vector<vi>& tbl, int nod, int steps){
bcc     rep(i,0,sz(tbl))
9c0         if(steps&(1<<i)) nod = tbl[i][nod];
3a8     return nod;
065 }

ea2 int lca(vector<vi>& tbl, vi& depth, int a, int b) {
a0c     if (depth[a] < depth[b]) swap(a, b);
4d1     a = jmp(tbl, a, depth[a] - depth[b]);
6df     if (a == b) return a;
586     for (int i = sz(tbl); i--;) {
cc4         int c = tbl[i][a], d = tbl[i][b];
872         if (c != d) a = c, b = d;
c29     }
c09     return tbl[0][a];
b5d }
```

LCA.h

Description: Data structure for computing lowest common ancestors in a tree (with 0 as root). C should be an adjacency list of the tree, either directed or undirected.

Time: $\mathcal{O}(N \log N + Q)$

"../data-structures/RMQ.h" 0f62fb, 21 lines

```
33e struct LCA {
d07     int T = 0;
f2e     vi time, path, ret;
4c7     RMQ<int> rmq;

764     LCA(vector<vi>& C) : time(sz(C)), rmq((dfs(C,0,-1), ret))
    {}
aaa     void dfs(vector<vi>& C, int v, int par) {
a0c         time[v] = T++;
e2a         for (int y : C[v]) if (y != par) {
cd1             path.push_back(v), ret.push_back(time[v]);
3d7             dfs(C, y, v);
bd2         }
f9a     }

7be     int lca(int a, int b) {
6df         if (a == b) return a;
394         tie(a, b) = minmax(time[a], time[b]);
8c2         return path[rmq.query(a, b)];
b82     }

//dist(a,b){return depth[a] + depth[b] - 2*depth[lca(a,b)
];}

0f6 };
```

CompressTree.h

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S| - 1$) pairwise LCA's and compressing edges. Returns a list of (par, orig.index) representing a tree rooted at 0. The root points to itself.

Time: $\mathcal{O}(|S| \log |S|)$

"LCA.h" 9775a0, 21 lines

```
6d4 typedef vector<pair<int, int>> vpi;
021 vpi compressTree(LCA& lca, const vi& subset) {
4ef     static vi rev; rev.resize(sz(lca.time));
b8b     vi li = subset, &T = lca.time;
4f8     auto cmp = [&](int a, int b) { return T[a] < T[b]; };
ff4     sort(all(li), cmp);
070     int m = sz(li)-1;
670     rep(i,0,m) {
882         int a = li[i], b = li[i+1];
9e6         li.push_back(lca.lca(a, b));
677     }
ff4     sort(all(li), cmp);
efc     li.erase(unique(all(li)), li.end());
17b     rep(i,0,sz(li)) rev[li[i]] = i;
993     vpi ret = {pii(0, li[0])};
6f3     rep(i,0,sz(li)-1) {
882         int a = li[i], b = li[i+1];
aa6         ret.emplace_back(rev[lca.lca(a, b)], b);
5ef     }
edf     return ret;
83c }
```

HLD.h

Description: Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most $\log(n)$ light edges. Code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. Takes as input the full adjacency list. VALS_EDGES being true means that values are stored in the edges, as opposed to the nodes. All values initialized to the segtree default. Root must be 0.

Time: $\mathcal{O}\left((\log N)^2\right)$

"../data-structures/LazySegmentTree.h" 9547af, 46 lines

```

303 template <bool VALS_EDGES> struct HLD {
0bf     int N, tim = 0;
698     vector<vi> adj;
1a3     vi par, siz, rt, pos;
0c0     Node *tree;
366     HLD(vector<vi> adj_)
78f         : N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1),
94d         rt(N), pos(N), tree(new Node(0, N)){ dfsSz(0); dfsHld(0)
; }
d38     void dfsSz(int v) {
ba3         for (int& u : adj[v]) {
07f             adj[u].erase(find(all(adj[u]), v));
187             par[u] = v;
a00             dfsSz(u);
a02             siz[v] += siz[u];
68d             if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
9f6         }
db8     }
7e1     void dfsHld(int v) {
06c         pos[v] = tim++;
f74         for (int u : adj[v]) {
c92             rt[u] = (u == adj[v][0] ? rt[v] : u);
9f9             dfsHld(u);
ee6         }
044     }
template <class B> void process(int u, int v, B op) {
205     for (;;) v = par[rt[v]] {
bdc         if (pos[u] > pos[v]) swap(u, v);
114         if (rt[u] == rt[v]) break;
052         op(pos[rt[v]], pos[v] + 1);
001     }
12c     op(pos[u] + VALS_EDGES, pos[v] + 1);
431     }
f41     void modifyPath(int u, int v, int val) {
f43         process(u, v, [&](int l, int r) { tree->add(l, r, val);
});
a18     }
c66     int queryPath(int u, int v) { // Modify depending on
problem
667         int res = -1e9;
227         process(u, v, [&](int l, int r) {
59e             res = max(res, tree->query(l, r));
b1d         });
b50         return res;
1a6     }
770     int querySubtree(int v) { // modifySubtree is similar
808         return tree->query(pos[v] + VALS_EDGES, pos[v] + siz[v])
;
e86     }
954 };

```

LinkCutTree.h

Description: Represents a forest of unrooted trees. You can add and remove edges (as long as the result is still a forest), and check whether two nodes are in the same tree.

Time: All operations take amortized $\mathcal{O}(\log N)$.

0fb462, 90 lines

```

bf2 struct Node { // Splay tree. Root's pp contains tree's
parent.
dc5     Node *p = 0, *pp = 0, *c[2];
d27     bool flip = 0;
d41     Node() { c[0] = c[1] = 0; fix(); }
01a     void fix() {
2ac         if (c[0]) c[0]->p = this;
6ee         if (c[1]) c[1]->p = this;
// (+ update sum of subtree elements etc. if wanted)
454     }
c3f     void pushFlip() {

```

```

042     if (!flip) return;
4cd     flip = 0; swap(c[0], c[1]);
4a1     if (c[0]) c[0]->flip ^= 1;
d21     if (c[1]) c[1]->flip ^= 1;
0cc     }
952     int up() { return p ? p->c[1] == this : -1; }
13b     void rot(int i, int b) {
95d         int h = i ^ b;
35e         Node *x = c[i], *y = b == 2 ? x : x->c[h], *z = b ? y :
x;
066         if ((y->p = p)) p->c[up(i)] = y;
7ee         c[i] = z->c[i ^ 1];
76b         if (b < 2) {
986             x->c[h] = y->c[h ^ 1];
4fe             y->c[h ^ 1] = x;
1a8         }
752         z->c[i ^ 1] = this;
b23         fix(); x->fix(); y->fix();
3db         if (p) p->fix();
01b         swap(pp, y->pp);
1cf     }
2e8     void splay() {
7d3         for (pushFlip(); p; ) {
501             if (p->p) p->p->pushFlip();
571             p->pushFlip(); pushFlip();
126             int c1 = up(), c2 = p->up();
b7b             if (c2 == -1) p->rot(c1, 2);
29e             else p->p->rot(c2, c1 != c2);
e63         }
bfb     }
fd7     Node* first() {
289         pushFlip();
5f4         return c[0] ? c[0]->first() : (splay(), this);
67f     }
225 };
efa struct LinkCut {
4a4     vector<Node> node;
373     LinkCut(int N) : node(N) {}
c63     void link(int u, int v) { // add an edge (u, v)
052         assert(!connected(u, v));
530         makeRoot(&node[u]);
28a         node[u].pp = &node[v];
607     }
684     void cut(int u, int v) { // remove an edge (u, v)
315         Node *x = &node[u], *top = &node[v];
336         makeRoot(top); x->splay();
1d6         assert(top == (x->pp ? x->c[0]));
71e         if (x->pp) x->pp = 0;
4e6         else {
a42             x->c[0] = top->p = 0;
bc2             x->fix();
8ac         }
a58     }
378     bool connected(int u, int v) { // are u, v in the same
tree?
d94         Node* nu = access(&node[u])->first();
132         return nu == access(&node[v])->first();
b80     }
6ab     void makeRoot(Node* u) {
29b         access(u);
f18         u->splay();
3d7         if (u->c[0]) {
563             u->c[0]->p = 0;
b89             u->c[0]->flip ^= 1;
0aa             u->c[0]->pp = u;
828             u->c[0] = 0;
1a0             u->fix();

```

```

586     }
74c     }
ef5     Node* access(Node* u) {
f18         u->splay();
072         while (Node* pp = u->pp) {
b8c             pp->splay(); u->pp = 0;
fec             if (pp->c[1]) {
1cc                 pp->c[1]->p = 0; pp->c[1]->pp = pp; }
1ad             pp->c[1] = u; pp->fix(); u = pp;
b10         }
03f         return u;
4ac     }
cea };

```

DirectedMST.h

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.

Time: $\mathcal{O}(E \log V)$

../data-structures/UnionFindRollback.h

39e620, 60 lines

```

030 struct Edge { int a, b; ll w; };
bf2 struct Node {
25f     Edge key;
c17     Node *l, *r;
981     ll delta;
a9c     void prop() {
6f9         key.w += delta;
d2d         if (l) l->delta += delta;
d86         if (r) r->delta += delta;
978         delta = 0;
0d3     }
866     Edge top() { prop(); return key; }
ab4 };
3eb     Node *merge(Node *a, Node *b) {
b9f         if (!a || !b) return a ? b;
626         a->prop(), b->prop();
dc2         if (a->key.w > b->key.w) swap(a, b);
485         swap(a->l, (a->r = merge(b, a->r)));
3f5         return a;
c51     }
7bb     void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

002 pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
8df     RollbackUF uf(n);
3f8     vector<Node*> heap(n);
563     for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node(e))
;
cd2     ll res = 0;
517     vi seen(n, -1), path(n), par(n);
559     seen[r] = r;
dd6     vector<Edge> Q(n), in(n, {-1,-1}), comp;
111     deque<tuple<int, int, vector<Edge>>> cys;
328     rep(s,0,n) {
3cb         int u = s, qi = 0, w;
a0a         while (seen[u] < 0) {
572             if (!heap[u]) return {-1,{};};
ebe             Edge e = heap[u]->top();
5ed             heap[u]->delta -= e.w, pop(heap[u]);
952             Q[qi] = e, path[qi++] = u, seen[u] = s;
d56             res += e.w, u = uf.find(e.a);
9e2             if (seen[u] == s) {
28d                 Node* cyc = 0;
cab                 int end = qi, time = uf.time();
f38                 do cyc = merge(cyc, heap[w = path[--qi]]);
4f9                 while (uf.join(u, w));
562                 u = uf.find(u), heap[u] = cyc, seen[u] = -1;
c06                 cys.push_front({u, time, {&Q[qi], &Q[end]}});
00a             }
c8f         }
068     rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];

```

```
fa3  }

e41  for (auto& [u,t,comp] : cycs) { // restore sol (optional)
36c    uf.rollback(t);
1d0    Edge inEdge = in[u];
251    for (auto& e : comp) in[uf.find(e.b)] = e;
56d    in[uf.find(inEdge.b)] = inEdge;
4f9  }
427  rep(i,0,n) par[i] = in[i].a;
efb  return {res, par};
efa }
```

7.8 Math

7.8.1 Number of Spanning Trees

Create an $N \times N$ matrix `mat`, and for each edge $a \rightarrow b \in G$, do `mat[a][b]--`, `mat[b][b]++` (and `mat[b][a]--`, `mat[a][a]++` if G is undirected). Remove the i th row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

7.8.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \dots \geq d_n$ exists iff $d_1 + \dots + d_n$ is even and for every $k = 1 \dots n$,

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k).$$

Geometry (8)

8.1 Geometric primitives

Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

47ec0a, 28 lines

```
48b template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
4fc template<class T>
f26 struct Point {
ea4     typedef Point P;
645     T x, y;
ea6     explicit Point(T x=0, T y=0) : x(x), y(y) {}
0d0     bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
ec7     bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
279     P operator+(P p) const { return P(x+p.x, y+p.y); }
40d     P operator-(P p) const { return P(x-p.x, y-p.y); }
e03     P operator*(T d) const { return P(x*d, y*d); }
0b9     P operator/(T d) const { return P(x/d, y/d); }
57b     T dot(P p) const { return x*p.x + y*p.y; }
460     T cross(P p) const { return x*p.y - y*p.x; }
b3f     T cross(P a, P b) const { return (a-*this).cross(b-*this); }
f68     T dist2() const { return x*x + y*y; }
18b     double dist() const { return sqrt((double)dist2()); }
// angle to x-axis in interval [-pi, pi]
907     double angle() const { return atan2(y, x); }
d06     P unit() const { return *this/dist(); } // makes dist()==1
200     P perp() const { return P(-y, x); } // rotates +90 degrees
852     P normal() const { return perp().unit(); }
```

// returns point rotated 'a' radians ccw around the origin

f23 P rotate(double a) const {

482 return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }

902 friend ostream& operator<<(ostream& os, P p) {

9a9 return os << "(" << p.x << ", " << p.y << ")"; }

d2d };

lineDistance.h

Description:

Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.

53630a, 4 lines

7dc template<class P>

da1 double lineDist(P a, P b, P p) {

e07 return (double) (b-a).cross(p-a) / (b-a).dist();

c8d }

SegmentDistance.h

Description:

Returns the shortest distance between point p and the line segment from point s to e.

Usage: Point<double> a, b(2,2), p(1,1); bool onSegment = segDist(a,b,p) < 1e-10;

d5f39f, 6 lines

626 typedef Point<double> P;

312 double segDist(P s, P e, P p) {

a44 if (s==e) return (p-s).dist();

f81 auto d = (e-s).dist2(), t = min(d,max(.0, (p-s).dot(e-s)));

2c1 return ((p-s)*d-(e-s)*t).dist()/d;

4b2 }

SegmentIntersection.h

Description:

If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.

Usage: vector<P> inter = segInter(s1,e1,s2,e2); if (sz(inter)==1) cout << "segments intersect at " << inter[0] << endl;

"Point.h", "OnSegment.h"

9d5f72, 13 lines

dae template<class P> vector<P> segInter(P a, P b, P c, P d) {

0b6 auto oa = c.cross(d, a), ob = c.cross(d, b),

318 oc = a.cross(b, c), od = a.cross(b, d);

// Checks if intersection is single non-endpoint point.

914 if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)

e5b return {(a * ob - b * oa) / (ob - oa)};

4c1 set<P> s;

ccb if (onSegment(c, d, a)) s.insert(a);

0ad if (onSegment(c, d, b)) s.insert(b);

3d8 if (onSegment(a, b, c)) s.insert(c);

2fa if (onSegment(a, b, d)) s.insert(d);

a35 return {all(s)};

9d5 }

lineIntersection.h

Description:

If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.

Usage: auto res = lineInter(s1,e1,s2,e2); if (res.first == 1) cout << "intersection point at " << res.second << endl;

"Point.h"

a01f81, 8 lines

7dc template<class P>

0bf pair<int, P> lineInter(P s1, P e1, P s2, P e2) {

14f auto d = (e1 - s1).cross(e2 - s2);

8cc if (d == 0) // if parallel

d99 return {-(s1.cross(e1, s2) == 0), P(0, 0)};

f6b auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);

9b8 return {1, (s1 * p + e1 * q) / d};

472 }

sideOf.h

Description:

Returns where p is as seen from s towards e. 1/0/-1 ⇔ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

Usage: bool left = sideOf(p1,p2,q)==1;

"Point.h"

ca4665, 9 lines

7dc template<class P>

70b int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }

sideOf.h

7dc template<class P>

521 int sideOf(P s, P e, P p, double eps) {

79e auto a = (e-s).cross(p-s);

653 double l = (e-s).dist()*eps;

c32 return (a > l) - (a < -l);

fbr }

OnSegment.h

Description:

Returns true iff p lies on the line segment from s to e. Use (segDist(s,e,p)<=epsilon) instead when using Point<double>.

"Point.h"

c597e8, 3 lines

514 template<class P> bool onSegment(P s, P e, P p) {

5fb return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;

c59 }

linearTransformation.h

Description:

Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.

"Point.h"

7871e9, 5 lines

626 typedef Point<double> P;

9ec P linearTransformation(P p0, P p1, P q0, P q1, P r) {

99f P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));

0aa return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();

4cb }

LineProjectionReflection.h

Description: Projects point p onto line ab. Set refl=true to get reflection of point p across line ab instead. The wrong point will be returned if P is an integer point and the desired point doesn't have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow.

"Point.h"	b5562d, 5 lines
7dc template <class P> 981 P lineProj(P a, P b, P p, bool refl= false) { de3 P v = b - a; 3fc return p - v.perp()*(1+refl)*v.cross(p-a)/v.dist2(); 4b7 }	

Angle.h

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.

Usage: vector<Angle> v = {w[0], w[0].t360() ...}; // sorted
int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }
// sweeps j such that (j-i) represents the number of positively oriented triangles with vertices at 0 and i

"Point.h"	b5562d, 5 lines
755 struct Angle { e91 int x, y; 8bd int t; 5ac Angle(int x, int y, int t=0) : x(x), y(y), t(t) {} de8 Angle operator -(Angle b) const { return {x-b.x, y-b.y, t}; } 3cd int half() const { 840 assert(x y); aa4 return y < 0 (y == 0 && x < 0); c93 } dfc Angle t90() const { return {-y, x, t + (half() && x >= 0)} ; } 726 Angle t180() const { return {-x, -y, t + half()}; } 925 Angle t360() const { return {x, y, t + 1}; } e25 }; a92 bool operator <(Angle a, Angle b) { // add a.dist2() and b.dist2() to also compare distances ea7 return make_tuple(a.t, a.half(), a.y * (1l)b.x) < 05f make_tuple(b.t, b.half(), a.x * (1l)b.y); ce5 }	

// Given two points, this calculates the smallest angle between
// them, i.e., the angle that covers the defined line segment.
908 pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
ee4 **if** (b < a) swap(a, b);
423 **return** (b < a.t180() ?
c35 make_pair(a, b) : make_pair(b, a.t360()));
5ea }
784 Angle **operator**+(Angle a, Angle b) { // point a + vector b
eb1 Angle r(a.x + b.x, a.y + b.y, a.t);
8ca **if** (a.t180() < r) r.t--;
d9f **return** r.t180() < a ? r.t360() : r;
3d8 }
106 Angle angleDiff(Angle a, Angle b) { // angle b - angle a
125 **int** tu = b.t - a.t; a.t = b.t;
e63 **return** {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b < a)
 };
ba3 }

8.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

"Point.h"	84d6d3, 11 lines
626 typedef Point< double > P; 27f bool circleInter(P a,P b, double r1, double r2,pair<P, P>* out) {	

```
b48   if (a == b) { assert(r1 != r2); return false; }  
f30   P vec = b - a;  
bd0   double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,  
270       p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*  
      d2;  
5b0   if (sum*sum < d2 || dif*dif > d2) return false;  
84d   P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) /  
      d2);  
21e   *out = {mid + per, mid - per};  
8a6   return true;  
c64 }
```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents - 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

"Point.h"	b0153d, 13 lines
7dc template <class P> 3a5 vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) { c0b P d = c2 - c1; 432 double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr; 018 if (d2 == 0 h2 < 0) return {}; c14 vector<pair<P, P>> out; 092 for (double sign : {-1, 1}) { 2ad P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2; 2e3 out.push_back({c1 + v * r1, c2 + v * r2}); e25 } b21 if (h2 == 0) out.pop_back(); fe8 return out; 483 }	

CircleLine.h

Description: Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points. P is intended to be Point<double>.

"Point.h"	e0cfba, 9 lines
7dc template <class P> 195 vector<P> circleLine(P c, double r, P a, P b) { 33b P ab = b - a, p = a + ab * (c-a).dot(ab) / ab.dist2(); 55a double s = a.cross(b, c), h2 = r*r - s*s / ab.dist2(); 3e4 if (h2 < 0) return {}; 071 if (h2 == 0) return {p}; 7cd P h = ab.unit() * sqrt(h2); d65 return {p - h, p + h}; 59a }	

CirclePolygonIntersection.h

Description: Returns the area of the intersection of a circle with a ccw polygon.

"Point.h"	19add1, 19 lines
626 typedef Point< double > P; 361 #define arg(p, q) atan2(p.cross(q), p.dot(q)) bb9 double circlePoly(P c, double r, vector<P> ps) { 6d1 auto tri = [&](P p, P q) { c9c auto r2 = r * r / 2; 291 P d = q - p; 127 auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2 ()); eea auto det = a * a - b; 691 if (det <= 0) return arg(p, q) * r2;	

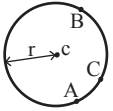
```
f43   auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det)  
      );  
aba   if (t < 0 || 1 <= s) return arg(p, q) * r2;  
57f   P u = p + d * s, v = q + d * (t-1);  
8c0   return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2;  
a52   };  
bef   auto sum = 0.0;  
8f4   rep(i,0,sz(ps))  
3b7   sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);  
e66   return sum;  
f08 }
```

circumcircle.h

Description:

The circumcirle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.

"Point.h"	6cffc5, 9 lines
626 typedef Point< double > P; 31a double ccRadius(P A, P B, P C) { 14b return (B-A).dist()*(C-B).dist()*(A-C).dist())/ f73 abs((B-A).cross(C-A))/2; 7db } f0c P ccCenter(P A, P B, P C) { 28a P b = C-A, c = B-A; 680 return A + (b*c.dist2()-(c*b.dist2()).perp()/b.cross(c)/2; 686 }	



MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points. **Time:** expected $\mathcal{O}(n)$

"circumcircle.h"	09dd0a, 17 lines
a28 pair<P, double > mec(vector<P> ps) { 4da shuffle(all(ps), mt19937(time(0))); f6a P o = ps[0]; 328 double r = 0, EPS = 1 + 1e-8; 2be rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) { 5cc o = ps[i], r = 0; 4da rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) { a30 o = (ps[i] + ps[j]) / 2; 6f7 r = (o - ps[i]).dist(); 102 rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) { fa9 o = ccCenter(ps[i], ps[j], ps[k]); 6f7 r = (o - ps[i]).dist(); 648 } 7b0 } dcf } 645 return {o, r}; 09d }	

8.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

Usage: vector<P> v = {P{4,4}, P{1,2}, P{2,1}};
bool in = inPolygon(v, P{3, 3}, false);
Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h"	2bf504, 11 lines
7dc template <class P> 0cc bool inPolygon(vector<P> &p, P a, bool strict = true) { 8b7 int cnt = 0, n = sz(p); fea rep(i,0,n) { 444 P q = p[(i + 1) % n]; cbd if (onSegment(p[i], q, a)) return !strict; //or: if (segDist(p[i], q, a) <= eps) return !strict;	

```
007     cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) >
0;
1b9   }
70a   return cnt;
c72 }
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

"Point.h"	f12300, 6 lines
4fc template <class T>	
a51 T polygonArea2(vector<Point<T>&& v) {	
2f8 T a = v.back().cross(v[0]);	
06e rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);	
3f5 return a;	
693 }	

PolygonCenter.h

Description: Returns the center of mass for a polygon.

Time: $\mathcal{O}(n)$

"Point.h"	9706dc, 9 lines
62e typedef Point<double> P;	
6d9 P polygonCenter(const vector<P>& v) {	
f9f P res(0, 0); double A = 0;	
70b for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {	
34e res = res + (v[i] + v[j]) * v[j].cross(v[i]);	
3ea A += v[j].cross(v[i]);	
307 }	
33c return res / A / 3;	
0d0 }	

PolygonCut.h

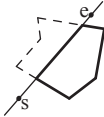
Description:

Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.

Usage: vector<P> p = ...;

p = polygonCut(p, P(0,0), P(1,0));

"Point.h"	d07181, 13 lines
62e typedef Point<double> P;	
37d vector<P> polygonCut(const vector<P>& poly, P s, P e) {	
fe2 vector<P> res;	
d48 rep(i,0,sz(poly)) {	
21c P cur = poly[i], prev = i ? poly[i-1] : poly.back();	
c5f auto a = s.cross(e, cur), b = s.cross(e, prev);	
2dc if ((a < 0) != (b < 0))	
380 res.push_back(cur + (prev - cur) * (a / (a - b)));	
c5c if (a < 0)	
a5f res.push_back(cur);	
757 }	
b50 return res;	
42c }	



PolygonUnion.h

Description: Calculates the area of the union of n polygons (not necessarily convex). The points within each polygon must be given in CCW order. (Epsilon checks may optionally be added to sideOf/sgn, but shouldn't be needed.)

Time: $\mathcal{O}(N^2)$, where N is the total number of points

"Point.h", "sideOf.h"	3931c6, 33 lines
62e typedef Point<double> P;	
142 double rat(P a, P b) { return sgn(b.x) ? a.x/b.x : a.y/b.y;	
}	
61d double polyUnion(vector<vector<P>>& poly) {	
499 double ret = 0;	
9af rep(i,0,sz(poly)) rep(v,0,sz(poly[i])) {	
9c8 P A = poly[i][v], B = poly[i][(v + 1) % sz(poly[i])];	
05c vector<pair<double, int>> segs = {{0, 0}, {1, 0}};	

cbd rep(j,0,sz(poly)) if (i != j) {	
cc1 rep(u,0,sz(poly[j])) {	
418 P C = poly[j][u], D = poly[j][(u + 1) % sz(poly[j])]	
};	
688 int sc = sideOf(A, B, C), sd = sideOf(A, B, D);	
68b if (sc != sd) {	
295 double sa = C.cross(D, A), sb = C.cross(D, B);	
e90 if (min(sc, sd) < 0)	
dac segs.emplace_back(sa / (sa - sb), sgn(sc - sd));	
cf7 } else if (!sc && !sd && j<i && sgn((B-A).dot(D-C))	
>0){	
5b4 segs.emplace_back(rat(C - A, B - A), 1);	
e96 segs.emplace_back(rat(D - A, B - A), -1);	
313 }	
0d1 }	
fdc }	
861 sort(all(segs));	
153 for (auto& s : segs) s.first = min(max(s.first, 0.0),	
1.0);	
68c double sum = 0;	
723 int cnt = segs[0].second;	
067 rep(j,1,sz(segs)) {	
081 if (!cnt) sum += segs[j].first - segs[j - 1].first;	
6e9 cnt += segs[j].second;	
f58 }	
320 ret += A.cross(B) * sum;	
191 }	
ad6 return ret / 2;	
6e8 }	

ConvexHull.h

Description:

Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.

Time: $\mathcal{O}(n \log n)$

"Point.h"	310954, 13 lines
2c0 typedef Point<ll> P;	
f16 vector<P> convexHull(vector<P> pts) {	
f78 if (sz(pts) <= 1) return pts;	
3cb sort(all(pts));	
abf vector<P> h(sz(pts)+1);	
573 int s = 0, t = 0;	
628 for (int it = 2; it--; s = --t, reverse(all(pts)))	
4eb for (P p : pts) {	
3da while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t	
--;	
f39 h[t++] = p;	
bf0 }	
036 return {h.begin(), h.begin() + t - (t == 2 && h[0] == h	
[1])};	
ec8 }	



HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).

Time: $\mathcal{O}(n)$

"Point.h"	c571b8, 12 lines
2c0 typedef Point<ll> P;	
d31 array<P, 2> hullDiameter(vector<P> S) {	
e79 int n = sz(S), j = n < 2 ? 0 : 1;	
354 pair<ll, array<P, 2>> res({0, {S[0], S[0]}});	
e4d rep(i,0,j)	
42e for (; j = (j + 1) % n) {	
ca1 res = max(res, {{S[i] - S[j]}.dist2(), {S[i], S[j]}});	
be8 if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >=	
0)	
c2b break;	

56c }	
3f2 return res.second;	
5f7 }	

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.

Time: $\mathcal{O}(\log N)$

"Point.h", "sideOf.h", "OnSegment.h"	71446b, 14 lines
2c0 typedef Point<ll> P;	
2d4 bool inHull(const vector<P>& l, P p, bool strict = true) {	
d44 int a = 1, b = sz(l) - 1, r = !strict;	
5cc if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);	
6bc if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);	
456 if (sideOf(l[0], l[a], p) >= r sideOf(l[0], l[b], p) <=	
-r)	
d1f return false;	
48a while (abs(a - b) > 1) {	
4f7 int c = (a + b) / 2;	
ac8 (sideOf(l[0], l[c], p) > 0 ? b : a) = c;	
b26 }	
06f return sgn(l[a].cross(l[b], p)) < r;	
c74 }	

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: $\bullet(-1, -1)$ if no collision, $\bullet(i, -1)$ if touching the corner i , $\bullet(i, i)$ if along side $(i, i + 1)$, $\bullet(i, j)$ if crossing sides $(i, i + 1)$ and $(j, j + 1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i, i + 1)$. The points are returned in the same order as the line hits the polygon. extrVertex returns the point of a hull with the max projection onto a line.

Time: $\mathcal{O}(\log n)$

"Point.h"	7cf45b, 39 lines
530 #define cmp(i, j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n	
]))	
f84 #define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0	
e7e template <class P> int extrVertex(vector<P>& poly, P dir) {	
747 int n = sz(poly), lo = 0, hi = n;	
fdf if (extr(0)) return 0;	
3d1 while (lo + 1 < hi) {	
591 int m = (lo + hi) / 2;	
855 if (extr(m)) return m;	
c0c int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);	
f48 (ls < ms (ls == ms && ls == cmp(lo, m)) ? hi : lo) =	
m;	
68a }	
253 return lo;	
7f0 }	
8e0 #define cmpL(i) sgn(a.cross(poly[i], b))	
7dc template <class P>	
ec4 array<int, 2> lineHull(P a, P b, vector<P>& poly) {	
409 int endA = extrVertex(poly, (a - b).perp());	
761 int endB = extrVertex(poly, (b - a).perp());	
1a8 if (cmpL(endA) < 0 cmpL(endB) > 0)	
423 return {-1, -1};	
649 array<int, 2> res;	
f4b rep(i,0,2) {	
234 int lo = endB, hi = endA, n = sz(poly);	
c2d while ((lo + 1) % n != hi) {	
57e int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;	
7fe (cmpL(m) == cmpL(endB) ? lo : hi) = m;	
525 }	
7dd res[i] = (lo + !cmpL(hi)) % n;	

```
35e swap(endA, endB);
c05 }
e00 if (res[0] == res[1]) return {res[0], -1};
3d1 if (!cmpL(res[0]) && !cmpL(res[1]))
959 switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
3f3 case 0: return {res[0], res[0]};
223 case 2: return {res[1], res[1]};
8fa }
b50 return res;
36f }
```

MinkowskiSum.h

Description: Computes the Minkowski sum C of two convex polygons A and B . It is proven that C is a convex polygon with at most $|A| + |B|$ vertices. Vertices of polygons A and B must be given in CCW order, and vertices of C will be returned in CCW order, similarly.
Time: $\mathcal{O}(|A| + |B|)$

"Point.h"	f85ee8, 23 lines
-----------	------------------

```
2c0 typedef Point<ll> P;
7b1 void reorder_polygon(vector<P>& a) {
1ba int p = 0;
af6 rep(i, 1, sz(a)) {
edf if (tie(a[i].y, a[i].x) < tie(a[p].y, a[p].x)) p = i;
db3 }
d2f rotate(a.begin(), a.begin() + p, a.end());
8ca }
c8c vector<P> minkowski(vector<P> a, vector<P> b) {
317 reorder_polygon(a);
83d reorder_polygon(b);
5ca a.push_back(a[0]); a.push_back(a[1]);
258 b.push_back(b[0]); b.push_back(b[1]);
035 vector<P> c;
692 int i = 0, j = 0;
ffb while (i < sz(a) - 2 || j < sz(b) - 2) {
018 c.push_back(a[i] + b[j]);
b36 auto cross = (a[i + 1] - a[i]).cross(b[j + 1] - b[j]);
7ed if (cross >= 0 && i < sz(a) - 2) ++i;
6df if (cross <= 0 && j < sz(b) - 2) ++j;
19f }
807 return c;
a46 }
```

8.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.
Time: $\mathcal{O}(n \log n)$

"Point.h"	ac41a6, 17 lines
-----------	------------------

```
2c0 typedef Point<ll> P;
24b pair<P, P> closest(vector<P> v) {
7f9 assert(sz(v) > 1);
7f7 set<P> S;
879 sort(all(v), [](P a, P b) { return a.y < b.y; });
571 pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
ee8 int j = 0;
813 for (P p : v) {
3fb P d{1 + (ll)sqrt(ret.first), 0};
8be while (v[j].y <= p.y - d.x) S.erase(v[j++]);
a5a auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d
);
c77 for (; lo != hi; ++lo)
113 ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
8aa S.insert(p);
5b0 }
70d return ret.second;
bf2 }
```

ManhattanMST.h

Description: Given N points, returns up to $4 \cdot N$ edges, which are guaranteed to contain a minimum spanning tree for the graph with edge weights $w(p, q) = -p.x - q.x - p.y - q.y$. Edges are in the form (distance, src, dst). Use a standard MST algorithm on the result to find the final MST.
Time: $\mathcal{O}(N \log N)$

"Point.h"	df6f59, 23 lines
-----------	------------------

```
bbe typedef Point<int> P;
ea9 vector<array<int, 3>> manhattanMST(vector<P> ps) {
850 vi id(sz(ps));
27c iota(all(id), 0);
8c1 vector<array<int, 3>> edges;
8de rep(k, 0, 4) {
1dd sort(all(id), [&](int i, int j) {
02b return (ps[i]-ps[j]).x < (ps[j]-ps[i]).y;});
702 map<int, int> sweep;
1e2 for (int i : id) {
84d for (auto it = sweep.lower_bound(-ps[i].y);
904 it != sweep.end(); sweep.erase(it++)) {
61d int j = it->second;
6f3 P d = ps[i] - ps[j];
d18 if (d.y > d.x) break;
537 edges.push_back({d.y + d.x, i, j});
271 }
923 sweep[-ps[i].y] = i;
e69 }
4eb for (P& p : ps) if (k & 1) p.x = -p.x; else swap(p.x, p.
y);
a11 }
da2 return edges;
a11 }
```

kdTree.h

Description: KD-tree (2d, can be extended to 3d)

"Point.h"	69ff55, 63 lines
-----------	------------------

```
9a6 typedef long long T;
293 typedef Point<T> P;
305 const T INF = numeric_limits<T>::max();

8fc bool on_x(P a, P b) { return a.x < b.x; }
308 bool on_y(P a, P b) { return a.y < b.y; }

bf2 struct Node {
975 P pt; // if this is a leaf, the single point in it
877 T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds
a23 Node *first = 0, *second = 0;

72a T distance(P p) { // min squared distance to a point
28b T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x);
88e T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y);
d98 return (P(x,y) - p).dist2();
af5 }

d97 Node(vector<P>&& vp) : pt(vp[0]) {
741 for (P p : vp) {
ad3 x0 = min(x0, p.x); x1 = max(x1, p.x);
e5d y0 = min(y0, p.y); y1 = max(y1, p.y);
310 }
994 if (vp.size() > 1) {
// split on x if width >= height (not ideal...)
9b7 sort(all(vp), x1 - x0 >= y1 - y0 ? on_x : on_y);
// divide by taking half the array for each child (
not
// best performance with many duplicates in the
middle)

0f9 int half = sz(vp)/2;
48e first = new Node({vp.begin(), vp.begin() + half});
902 second = new Node({vp.begin() + half, vp.end()});
```

```
66e }
204 }
51c };

dad struct KDTree {
95f Node* root;
c30 KDTree(const vector<P>& vp) : root(new Node({all(vp)})) {}

0a9 pair<T, P> search(Node *node, P p) {
ec4 if (!node->first) {
// uncomment if we should not find the point itself:
// if (p == node->pt) return {INF, P()};
47e return make_pair((p - node->pt).dist2(), node->pt);
119 }

ea4 Node *f = node->first, *s = node->second;
d40 T bfirst = f->distance(p), bsec = s->distance(p);
a16 if (bfirst > bsec) swap(bsec, bfirst), swap(f, s);

// search closest side first, other side if needed

86c auto best = search(f, p);
314 if (bsec < best.first)
509 best = min(best, search(s, p));
f26 return best;
2b3 }

// find nearest point to a point, and its squared
distance
// (requires an arbitrary operator< for Point)
106 pair<T, P> nearest(P p) {
195 return search(root, p);
12b }
dc0 };
```

DelaunayTriangulation.h

Description: Computes the Delaunay triangulation of a set of points. Each circumcircle contains none of the input points. If any three points are collinear or any four are on the same circle, behavior is undefined.
Time: $\mathcal{O}(n^2)$

"Point.h", "3dHull.h"	c0e7bc, 10 lines
-----------------------	------------------

```
6ab template<class P, class F>
ff3 void delaunay(vector<P>& ps, F trifun) {
007 if (sz(ps) == 3) { int d = (ps[0].cross(ps[1], ps[2]) < 0)
;
645 trifun(0,1+d,2-d); }
568 vector<P3> p3;
825 for (P p : ps) p3.emplace_back(p.x, p.y, p.dist2());
08a if (sz(ps) > 3) for(auto t:hull13d(p3)) if ((p3[t.b]-p3[t.a
])).
566 cross(p3[t.c]-p3[t.a]).dot(P3(0,0,1)) < 0)
150 trifun(t.a, t.c, t.b);
8bb }
```

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], ... }, all counter-clockwise.
Time: $\mathcal{O}(n \log n)$

"Point.h"	cefd5f, 88 lines
-----------	------------------

```
2c0 typedef Point<ll> P;
806 typedef struct Quad* Q;
449 typedef __int128_t ll1; // (can be ll if coords are < 2e4)
59b P arb(LLONG_MAX, LLONG_MAX); // not equal to any other point

070 struct Quad {
461 Q rot, o; P p = arb; bool mark;
```

```

b38 P& F() { return r()->p; }
23a Q& r() { return rot->rot; }
f4f Q prev() { return rot->o->rot; }
57e Q next() { return r()->prev(); }
180 } *H;

d15 bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
4b4 lll p2 = p.dist2(), A = a.dist2()-p2,
ffa B = b.dist2()-p2, C = c.dist2()-p2;
59a return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B > 0;
6af }
00a Q makeEdge(P orig, P dest) {
bdf Q r = H ? H : new Quad{new Quad{new Quad{new Quad{}}}};
516 H = r->o; r->r()->r() = r;
2c3 rep(i,0,4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r()
();
ed2 r->p = orig; r->F() = dest;
4c1 return r;
b3b }
d8d void splice(Q a, Q b) {
686 swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
86c }
e92 Q connect(Q a, Q b) {
fc2 Q q = makeEdge(a->F(), b->p);
6e6 splice(q, a->next());
642 splice(q->r(), b);
bef return q;
4a4 }

196 pair<Q,Q> rec(const vector<P>& s) {
e63 if (sz(s) <= 3) {
4a0 Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back())
;
2ba if (sz(s) == 2) return { a, a->r() };
19e splice(a->r(), b);
5f8 auto side = s[0].cross(s[1], s[2]);
b9f Q c = side ? connect(b, a) : 0;
348 return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
c9e }

5ef #define H(e) e->F(), e->p
c98 #define valid(e) (e->F().cross(H(base)) > 0)
a3e Q A, B, ra, rb;
f5e int half = sz(s) / 2;
391 tie(ra, A) = rec({all(s) - half});
49b tie(B, rb) = rec({sz(s) - half + all(s)});
f80 while ((B->p.cross(H(A)) < 0 && (A = A->next()) ||
b08 (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
76d Q base = connect(B->r(), A);
87f if (A->p == ra->p) ra = base->r();
b58 if (B->p == rb->p) rb = base;

3e6 #define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
f02 while (circ(e->dir->F(), H(base), e->F())) { \
936 Q t = e->dir; \
6d3 splice(e, e->prev()); \
16e splice(e->r(), e->r()->prev()); \
d47 e->o = H; H = e; e = t; \
a2e }
1de for (;;) {
eaa DEL(LC, base->r(), o); DEL(RC, base, prev());
6fa if (!valid(LC) && !valid(RC)) break;
e09 if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
b74 base = connect(RC, base->r());
295 else
271 base = connect(base->r(), LC->r());
fcf }
345 return { ra, rb };

```

```

7cf }

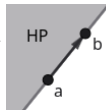
da1 vector<P> triangulate(vector<P> pts) {
af6 sort(all(pts)); assert(unique(all(pts)) == pts.end());
e00 if (sz(pts) < 2) return {};
235 Q e = rec(pts).first;
50c vector<Q> q = {e};
6c1 int qi = 0;
7a5 while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
806 #define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p)
; \
43e q.push_back(c->r()); c = c->next(); } while (c != e); }
9d6 ADD; pts.clear();
b58 while (qi < sz(q)) if (!(e = q[qi++])->mark) ADD;
a42 return pts;
a02 }

```

HalfPlaneIntersection.h

Description:

Computes the intersection of n half-planes. Each half-plane is represented as the region to the left side of a vector $\vec{ab}$. If the half-planes are already given sorted by angle, the sort in the first line of hpi can be removed, making the time $O(n)$.



Time: $O(n \log n)$

```

"Point.h" 99c1df, 43 lines
236 typedef long double dbl;
e9b typedef Point<dbl> P;
650 const dbl eps = 1e-9;
f74 struct HP {
812 P p, pq;
47f dbl ang;
6cc HP() {}
065 HP(const P& a, const P& b): p(a), pq(b - a) {
2a5 ang = atan2(pq.y, pq.x);
3af }
8ce bool out(const P& r) { return pq.cross(r - p) < -eps; }
396 bool operator<(const HP& e) const { return ang < e.ang; }
fb2 friend P inter(const HP& s, const HP& t) {
ccc dbl alpha = (t.p-s.p).cross(t.pq) / s.pq.cross(t.pq);
93b return s.p + (s.pq * alpha);
604 }
0cc };
2d4 vector<P> hpi(vector<HP>& H) {
f1a sort(all(H)); // can be removed if HPs are already sorted
ad3 deque<HP> dq;
486 int len = 0;
b09 rep(i, 0, sz(H)) {
6d0 while (len > 1 && H[i].out(inter(dq[len-1], dq[len-2])))
cfb dq.pop_back(), --len;
45a while (len > 1 && H[i].out(inter(dq[0], dq[1])))
21b dq.pop_front(), --len;
594 if (len > 0 && fabs1(H[i].pq.cross(dq[len-1].pq)) < eps)
{
48f if (H[i].pq.dot(dq[len-1].pq) < 0.0) return vector<P>
>();
372 if (H[i].out(dq[len-1].p)) dq.pop_back(), --len;
64e else continue;
2f7 }
0c2 dq.push_back(H[i]), ++len;
986 }
f09 while (len > 2 && dq[0].out(inter(dq[len-1], dq[len-2])))
cfb dq.pop_back(), --len;
f57 while (len > 2 && dq[len-1].out(inter(dq[0], dq[1])))
21b dq.pop_front(), --len;
1a3 if (len < 3) return vector<P>();
7e7 vector<P> ret(len);
702 rep(i, 0, len-1) ret[i] = inter(dq[i], dq[i+1]);

```

```

4fd ret.back() = inter(dq[len-1], dq[0]);
edf return ret;
8bc }

```

8.5 3D

PolyhedronVolume.h

Description: Magic formula for the volume of a polyhedron. Faces should point outwards.

3058c3, 6 lines

```

f9c template<class V, class L>
cb3 double signedPolyVolume(const V& p, const L& trilst) {
9e8 double v = 0;
b72 for (auto i : trilst) v += p[i.a].cross(p[i.b]).dot(p[i.c
]);
fb8 return v / 6;
fca }

```

Point3D.h

Description: Class to handle points in 3D space. T can be e.g. double or long long.

a94b7c, 31 lines

```

f10 template<class T> struct Point3D {
f07 typedef Point3D P;
329 T x, y, z;
cf2 explicit Point3D(T x=0, T y=0, T z=0) : x(x), y(y), z(z) {
}
007 bool operator<(P p) const {
290 return tie(x, y, z) < tie(p.x, p.y, p.z); }
bbf bool operator==(P p) const {
d7c return tie(x, y, z) == tie(p.x, p.y, p.z); }
d91 P operator+(P p) const { return P(x+p.x, y+p.y, z+p.z); }
674 P operator-(P p) const { return P(x-p.x, y-p.y, z-p.z); }
743 P operator*(T d) const { return P(x*d, y*d, z*d); }
17b P operator/(T d) const { return P(x/d, y/d, z/d); }
be8 T dot(P p) const { return x*p.x + y*p.y + z*p.z; }
28d P cross(P p) const {
923 return P(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x);
9ff }
b70 T dist2() const { return x*x + y*y + z*z; }
18b double dist() const { return sqrt((double)dist2()); }
//Azimuthal angle (longitude) to x-axis in interval [-pi,
pi]
3d6 double phi() const { return atan2(y, x); }
//Zenith angle (latitude) to the z-axis in interval [0,
pi]
0fa double theta() const { return atan2(sqrt(x*x+y*y),z); }
55e P unit() const { return *this/(T)dist(); } //makes dist()
=1
//returns unit vector normal to *this and p
685 P normal(P p) const { return cross(p).unit(); }
//returns point rotated 'angle' radians ccw around axis
c67 P rotate(double angle, P axis) const {
7cd double s = sin(angle), c = cos(angle); P u = axis.unit()
;
6b7 return u.dot(u)*(1-c) + (*this)*c - cross(u)*s;
73a }
a94 };

```

3dHull.h

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards.

Time: $O(n^2)$

5b45fc, 49 lines

```

"Point3D.h"
b8e typedef Point3D<double> P3;

9ce struct PR {
1fc void ins(int x) { (a == -1 ? a : b) = x; }

```

```
82f void rem(int x) { (a == x ? a : b) = -1; }
2ad int cnt() { return (a != -1) + (b != -1); }
ba2 int a, b;
cf7 };

5e4 struct F { P3 q; int a, b, c; };

b6d vector<F> hull3d(const vector<P3>& A) {
cd9 assert(sz(A) >= 4);
ec1 vector<vector<PR>> E(sz(A), vector<PR>(sz(A), {-1, -1}));
394 #define E(x,y) E[f.x][f.y]
af6 vector<F> FS;
9e0 auto mf = [&](int i, int j, int k, int l) {
2ce P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
fa1 if (q.dot(A[l]) > q.dot(A[i]))
eaa q = q * -1;
f22 F f{q, i, j, k};
ee5 E(a,b).ins(k); E(a,c).ins(j); E(b,c).ins(i);
471 FS.push_back(f);
d73 };
30c rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4)
047 mf(i, j, k, 6 - i - j - k);

3ef rep(i,4,sz(A)) {
3b5 rep(j,0,sz(FS)) {
068 F f = FS[j];
04f if (f.q.dot(A[i]) > f.q.dot(A[f.a])) {
412 E(a,b).rem(f.c);
b61 E(a,c).rem(f.b);
e5c E(b,c).rem(f.a);
8d5 swap(FS[j--], FS.back());
eef FS.pop_back();
5cd }
220 }
97f int nw = sz(FS);
ce3 rep(j,0,nw) {
068 F f = FS[j];
561 #define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f.
c);
3da C(a, b, c); C(a, c, b); C(b, c, a);
248 }
472 }
864 for (F& it : FS) if ((A[it.b] - A[it.a]).cross(
770 A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b);
311 return FS;
be2 };
```

sphericalDistance.h

Description: Returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) f1 (φ₁) and f2 (φ₂) from x axis and zenith angles (latitude) t1 (θ₁) and t2 (θ₂) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. dx*radius is then the difference between the two points in the x direction and d*radius is the total distance between the points.

611f07, 8 lines

```
c5f double sphericalDistance(double f1, double t1,
3e8 double f2, double t2, double radius) {
284 double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
277 double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
c7e double dz = cos(t2) - cos(t1);
c09 double d = sqrt(dx*dx + dy*dy + dz*dz);
154 return radius*2*asin(d/2);
4fa }
```

KMP.h

Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -> 0010123). Can be used to find all occurrences of a string.

Time: $\mathcal{O}(n)$

d4375c, 16 lines

```
132 vi pi(const string& s) {
7c1 vi p(sz(s));
1c5 rep(i,1,sz(s)) {
052 int g = p[i-1];
6c0 while (g && s[i] != s[g]) g = p[g-1];
7cf p[i] = g + (s[i] == s[g]);
e2f }
74e return p;
9cb }
```

218 vi match(const string& s, const string& pat) {
2fe vi p = pi(pat + '\0' + s), res;
3e3 rep(i,sz(p)-sz(s),sz(p))
c3d if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
b50 return res;
a23 }

Zfunc.h

Description: z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)

Time: $\mathcal{O}(n)$

ee09e2, 12 lines

```
44d vi Z(const string& S) {
11c vi z(sz(S));
4b6 int l = -1, r = -1;
c7f rep(i,1,sz(S)) {
4ab z[i] = i >= r ? 0 : min(r - i, z[i - 1]);
065 while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
9fa z[i]++;
fe0 if (i + z[i] > r)
a11 l = i, r = i + z[i];
44b }
070 return z;
ee0 }
```

Manacher.h

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down).

Time: $\mathcal{O}(N)$

e7ad79, 13 lines

```
fc1 array<vi, 2> manacher(const string& s) {
f89 int n = sz(s);
f77 array<vi,2> p = {vi(n+1), vi(n)};
c9a rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
24e int t = r-i!z;
e70 if (i<r) p[z][i] = min(t, p[z][l+t]);
ff6 int L = i-p[z][i], R = i+p[z][i]-!z;
649 while (L>=1 && R+1<n && s[L-1] == s[R+1])
895 p[z][i]++, L--, R++;
f28 if (R>r) l=L, r=R;
a84 }
74e return p;
e7a }
```

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.

Usage: rotate(v.begin(), v.begin()+minRotation(v), v.end());

Time: $\mathcal{O}(N)$

d07a42, 8 lines

```
5fa int minRotation(string s) {
c6c int a=0, N=sz(s); s += s;
840 rep(b,0,N) rep(k,0,N) {
```

```
873 if (a+k == b || s[a+k] < s[b+k]) {b += max(0, k-1);
break;}
068 if (s[a+k] > s[b+k]) { a = b; break; }
937 }
3f5 return a;
d07 }
```

SuffixArray.h

Description: Builds suffix array for a string. sa[i] is the starting index of the suffix which is i'th in the sorted suffix array. The returned vector is of size n + 1, and sa[0] = n. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: lcp[i] = lcp(sa[i], sa[i-1]), lcp[0] = 0. The input string must not contain any nul chars.

Time: $\mathcal{O}(n \log n)$

635552, 22 lines

```
3f4 struct SuffixArray {
ff9 vi sa, lcp;
d91 SuffixArray(string s, int lim=256) { // or vector<int>
d2c s.push_back(0); int n = sz(s), k = 0, a, b;
4af vi x(all(s)), y(n), ws(max(n, lim));
af4 sa = lcp = y, iota(all(sa), 0);
25d for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p
) {
3cd p = j, iota(all(y), n - j);
a02 rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;
911 fill(all(ws), 0);
55c rep(i,0,n) ws[x[i]]++;
c82 rep(i,1,lim) ws[i] += ws[i - 1];
a9e for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
c7d swap(x, y), p = 1, x[sa[0]] = 0;
e53 rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =
da0 (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p
++;
83b }
65b for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
904 for (k && k--, j = sa[x[i] - 1];
262 s[i + k] == s[j + k]; k++);
48f }
635 };
```

SuffixTree.h

Description: Ukkonen's algorithm for online suffix tree construction. Each node contains indices [l, r] into the string, and a list of child nodes. Suffixes are given by traversals of this tree, joining [l, r) substrings. The root is 0 (has l = -1, r = 0), non-existent children are -1. To get a complete tree, append a dummy symbol – otherwise it may contain an incomplete path (still useful for substring matching, though).

Time: $\mathcal{O}(26N)$

aae0b8, 50 lines

```
3a1 struct SuffixTree {
748 enum { N = 200010, ALPHA = 26 }; // N ~ 2*maxlen+10
d9f int toi(char c) { return c - 'a'; }
905 string a; // v = cur node, q = cur position
34d int t[N][ALPHA], l[N], r[N], p[N], s[N], v=0, q=0, m=2;

3cd void ukkadd(int i, int c) { suff:
04c if (r[v]<=q) {
7aa if (t[v][c]==-1) { t[v][c]=m; l[m]=i;
3e8 p[m++]=v; v=s[v]; q=r[v]; goto suff; }
4a1 v=t[v][c]; q=l[v];
690 }
4f5 if (q==-1 || c==toi(a[q])) q++; else {
305 l[m+1]=i; p[m+1]=m; l[m]=l[v]; r[m]=q;
bec p[m]=p[v]; t[m][c]=m+1; t[m][toi(a[q])]=v;
6cd l[v]=q; p[v]=m; t[p[m]][toi(a[l[m]])]=m;
c79 v=s[p[m]]; q=l[m];
ce4 while (q<r[m]) { v=t[v][toi(a[q])]; q+=r[v]-l[v]; }
147 if (q==r[m]) s[m]=v; else s[m]=m+2;
604 q=r[v]-(q-r[m]); m+=2; goto suff;
```

```
7c0     }
89a     }

731 SuffixTree(string a) : a(a) {
a17     fill(r,r+N,sz(a));
7f1     memset(s, 0, sizeof s);
6c7     memset(t, -1, sizeof t);
4d5     fill(t[1],t[1]+ALPHA,0);
2bf     s[0] = 1; l[0] = l[1] = -1; r[0] = r[1] = p[0] = p[1] =
0;
ec0     rep(i,0,sz(a)) ukkadd(i, toi(a[i]));
c40     }

// example: find longest common substring (uses ALPHA =
28)
d50 pii best;
5f9 int lcs(int node, int i1, int i2, int olen) {
01f     if (l[node] <= i1 && i1 < r[node]) return 1;
1fe     if (l[node] <= i2 && i2 < r[node]) return 2;
1d2     int mask = 0, len = node ? olen + (r[node] - l[node]) :
0;
932     rep(c,0,ALPHA) if (t[node][c] != -1)
8c7         mask |= lcs(t[node][c], i1, i2, len);
bcf     if (mask == 3)
59b         best = max(best, {len, r[node] - len});
65e     return mask;
cc3     }
ed4     static pii LCS(string s, string t) {
800         SuffixTree st(s + (char)('z' + 1) + t + (char)('z' + 2))
;
55e         st.lcs(0, sz(s), sz(s) + 1 + sz(t), 0);
204         return st.best;
39f     }
aae     };
```

Hashing.h

Description: Self-explanatory methods for string hashing.2d2a67, 44 lines

```
// Arithmetic mod 2^64-1. 2x slower than mod 2^64 and more
// code, but works on evil test data (e.g. Thue-Morse, where
// ABBA... and BAAB... of length 2^10 hash the same mod 2^64).
// "typedef ull H;" instead if you think test data is random,
// or work mod 10^9+7 if the Birthday paradox is not a problem.
41d typedef uint64_t ull;
54d struct H {
858     ull x; H(ull x=0) : x(x) {}
c9b     H operator+(H o) { return x + o.x + (x + o.x < x); }
5cd     H operator-(H o) { return *this + ~o.x; }
de0     H operator*(H o) { auto m = (__uint128_t)x * o.x;
681         return H((ull)m) + (ull)(m >> 64); }
bf2     ull get() const { return x + !~x; }
03c     bool operator==(H o) const { return get() == o.get(); }
0ab     bool operator<(H o) const { return get() < o.get(); }
bf6     };
862     static const H C = (1l)1e11+3; // (order ~ 3e9; random also
ok)

814 struct HashInterval {
588     vector<H> ha, pw;
aaa     HashInterval(string& str) : ha(sz(str)+1), pw(ha) {
523         pw[0] = 1;
7b2         rep(i,0,sz(str))
e95             ha[i+1] = ha[i] * C + str[i],
b86             pw[i+1] = pw[i] * C;
b90     }
a56     H hashInterval(int a, int b) { // hash [a, b)
5db         return ha[b] - ha[a] * pw[b - a];
664     }
122     };
```

```
45b vector<H> getHashes(string& str, int length) {
163     if (sz(str) < length) return {};
a7b     H h = 0, pw = 1;
0a6     rep(i,0,length)
c76         h = h * C + str[i], pw = pw * C;
9c1     vector<H> ret = {h};
696     rep(i,length,sz(str)) {
993         ret.push_back(h = h * C + str[i] - pw * str[i-length]);
6c8     }
edf     return ret;
aaa     }

553 H hashString(string& s){H h{}; for(char c:s) h=h*C+c;return
h;}
```

AhoCorasick.h

Description: Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(−, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries. Time: construction takes $\mathcal{O}(26N)$, where N = sum of length of patterns. find(x) is $\mathcal{O}(N)$, where N = length of x. findAll is $\mathcal{O}(NM)$.

f35677, 66 lines

```
51f struct AhoCorasick {
996     enum {alpha = 26, first = 'A'}; // change this!
bf2     struct Node {
// (nmatches is optional)
b92         int back, next[alpha], start = -1, end = -1, nmatches =
0;
e72         Node(int v) { memset(next, v, sizeof(next)); }
503     };
716     vector<Node> N;
e14     vi backp;
c33     void insert(string& s, int j) {
c74         assert(!s.empty());
443         int n = 0;
b4f         for (char c : s) {
ac1             int& m = N[n].next[c - first];
ca9             if (m == -1) { n = m = sz(N); N.emplace_back(-1); }
7af             else n = m;
366         }
e07         if (N[n].end == -1) N[n].start = j;
a93         backp.push_back(N[n].end);
ff4         N[n].end = j;
b4a         N[n].nmatches++;
0bb     }
735     AhoCorasick(vector<string>& pat) : N(1, -1) {
fdd         rep(i,0,sz(pat)) insert(pat[i], i);
1cf         N[0].back = sz(N);
fc3         N.emplace_back(0);

26a
f47         queue<int> q;
for (q.push(0); !q.empty(); q.pop()) {
7a6             int n = q.front(), prev = N[n].back;
798             rep(i,0,alpha) {
c6c                 int &ed = N[n].next[i], y = N[prev].next[i];
01b                 if (ed == -1) ed = y;
4e6                 else {
736                     N[ed].back = y;
1e8                     (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
61b                     = N[y].end;
5ef                     N[ed].nmatches += N[y].nmatches;
acb                     q.push(ed);
bc6                 }
}
```

```
ala     }
67c     }
cb0     }
860     vi find(string word) {
443         int n = 0;
e3a         vi res; // ll count = 0;
1c3         for (char c : word) {
6fb             n = N[n].next[c - first];
384             res.push_back(N[n].end);
// count += N[n].nmatches;

a03         }
b50         return res;
8c0     }
96f     vector<vi> findAll(vector<string>& pat, string word) {
dba         vi r = find(word);
a9f         vector<vi> res(sz(word));
c40         rep(i,0,sz(word)) {
c77             int ind = r[i];
c23             while (ind != -1) {
51d                 res[i - sz(pat[ind]) + 1].push_back(ind);
3fe                 ind = backp[ind];
ef9             }
a78         }
b50         return res;
a03     }
f35     };
```

Various (10)

10.1 Intervals

IntervalContainer.h

Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive). Time: $\mathcal{O}(\log N)$

edce47, 23 lines

```
d91 set<pii>::iterator addInterval(set<pii>& is, int L, int R) {
bb3     if (L == R) return is.end();
d4c     auto it = is.lower_bound({L, R}), before = it;
dc6     while (it != is.end() && it->first <= R) {
164         R = max(R, it->second);
1a5         before = it = is.erase(it);
fe9     }
1af     if (it != is.begin() && (--it)->second >= L) {
3ca         L = min(L, it->first);
164         R = max(R, it->second);
861         is.erase(it);
0de     }
aa0     return is.insert(before, {L,R});
d57     }

675 void removeInterval(set<pii>& is, int L, int R) {
17b     if (L == R) return;
bef     auto it = addInterval(is, L, R);
e14     auto r2 = it->second;
655     if (it->first == L) is.erase(it);
016     else (int&)it->second = L;
ee9     if (R != r2) is.emplace(R, r2);
059     }
```

IntervalCover.h

Description: Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change (A) to add || R.empty(). Returns empty set on failure (or if G is empty). Time: $\mathcal{O}(N \log N)$

9e9d8d, 19 lines

```
4fc template<class T>
d8e vi cover(pair<T, T> G, vector<pair<T, T>> I) {
3d5     vi S(sz(I)), R;
d00     iota(all(S), 0);
591     sort(all(S), [&](int a, int b) { return I[a] < I[b]; });
d10     T cur = G.first;
05e     int at = 0;
33e     while (cur < G.second) { // (A)
438         pair<T, int> mx = make_pair(cur, -1);
f07         while (at < sz(I) && I[S[at]].first <= cur) {
032             mx = max(mx, make_pair(I[S[at]].second, S[at]));
69a             at++;
c42         }
c54         if (mx.second == -1) return {};
953         cur = mx.first;
bf6         R.push_back(mx.second);
dd1     }
b1a     return R;
b8d }
```

ConstantIntervals.h

Description: Split a monotone function on [from, to) into a minimal set of half-open intervals on which it has the same value. Runs a callback g for each such interval.

Usage: constantIntervals(0, sz(v), [&](int x){return v[x];},
 [&](int lo, int hi, T val){...});
Time: $\mathcal{O}(k \log \frac{n}{k})$

753a4c, 19 lines

```
fb4 template<class F, class G, class T>
d89 void rec(int from, int to, F& f, G& g, int& i, T& p, T q) {
6b6     if (p == q) return;
329     if (from == to) {
9a3         g(i, to, p);
c80         i = to; p = q;
956     } else {
0e5         int mid = (from + to) >> 1;
96c         rec(from, mid, f, g, i, p, f(mid));
695         rec(mid+1, to, f, g, i, p, q);
eff     }
fb5 }
f07 template<class F, class G>
06c void constantIntervals(int from, int to, F f, G g) {
783     if (to <= from) return;
522     int i = from; auto p = f(i), q = f(to-1);
691     rec(from, to-1, f, g, i, p, q);
b80     g(i, to, q);
8bf }
```

10.2 Misc. algorithms

TernarySearch.h

Description: Find the smallest i in [a,b] that maximizes $f(i)$, assuming that $f(a) < \dots < f(i) \geq \dots \geq f(b)$. To reverse which of the sides allows non-strict inequalities, change the < marked with (A) to <=, and reverse the loop at (B). To minimize f , change it to >, also at (B).

Usage: int ind = ternSearch(0,n-1,[&](int i){return a[i];});
Time: $\mathcal{O}(\log(b-a))$

9155b4, 11 lines

```
044 template<class F>
20f int ternSearch(int a, int b, F f) {
25b     assert(a <= b);
329     while (b - a >= 5) {
924         int mid = (a + b) / 2;
c9e         if (f(mid) < f(mid+1)) a = mid; // (A)
ceb         else b = mid+1;
ce7     }
95e     rep(i,a+1,b+1) if (f(a) < f(i)) a = i; // (B)
3f5     return a;
5d6 }
```

LIS.h

Description: Compute indices for the longest increasing subsequence.
Time: $\mathcal{O}(N \log N)$

2932a0, 17 lines

```
8d3 template<class I> vi lis(const vector<I>& S) {
173     if (S.empty()) return {};
1d7     vi prev(sz(S));
085     typedef pair<I, int> p;
249     vector<p> res;
897     rep(i,0,sz(S)) {
        // change 0 -> i for longest non-decreasing subsequence
b69         auto it = lower_bound(all(res), p{S[i], 0});
ef6         if (it == res.end()) res.emplace_back(), it = res.end()
-1;
df4         *it = {S[i], i};
6ce         prev[i] = it == res.begin() ? 0 : (it-1)->second;
147     }
629     int L = sz(res), cur = res.back().second;
bf5     vi ans(L);
ade     while (L--) ans[L] = cur, cur = prev[cur];
ba7     return ans;
293 }
```

FastKnapsack.h

Description: Given N non-negative integer weights w and a non-negative target t, computes the maximum $S \leq t$ such that S is the sum of some subset of the weights.
Time: $\mathcal{O}(N \max(w_i))$

b20ccc, 16 lines

```
4d3 int knapsack(vi w, int t) {
9af     int a = 0, b = 0, x;
50d     while (b < sz(w) && a + w[b] <= t) a += w[b++];
c8b     if (b == sz(w)) return a;
2b8     int m = *max_element(all(w));
754     vi u, v(2*m, -1);
0a2     v[a+m-t] = b;
564     rep(i,b,sz(w)) {
a68         u = v;
052         rep(x,0,m) v[x+w[i]] = max(v[x+w[i]], u[x]);
605         for (x = 2*m; --x > m;) rep(j, max(0,u[x]), v[x])
a42             v[x-w[j]] = max(v[x-w[j]], j);
ac5     }
4de     for (a = t; v[a+m-t] < 0; a--);
3f5     return a;
b20 }
```

SumOverSubsets.h

Description: Given an array a with 2^n elements, computes $f(x)$ for all $i = 0, 1, 2, \dots, 2^n - 1$. Here, $f(x)$ represents the sum of values in array a for all subsets i of x . That is, $f(x) = \sum_{i \subseteq x} a[i]$.

79aab5, 7 lines

```
20a vector<ll> sos(vector<ll> a) {
121     int n = __lg(sz(a));
0c6     rep(i, 0, n) rep(x, 0, 1 << n) {
7ae         if (x & (1 << i)) a[x] += a[x ^ (1 << i)];
93b     }
3f5     return a;
79a }
```

10.3 Dynamic programming

KnuthDP.h

Description: When doing DP on intervals: $a[i][j] = \min_{i < k < j} (a[i][k] + a[k][j]) + f(i, j)$, where the (minimal) optimal k increases with both i and j , one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j-1]$ and $p[i+1][j]$. This is known as Knuth DP. Sufficient criteria for this are if $f(b, c) \leq f(a, d)$ and $f(a, c) + f(b, d) \leq f(a, d) + f(b, c)$ for all $a \leq b \leq c \leq d$. Consider also: LineContainer (ch. Data structures), monotone queues, ternary search.

Time: $\mathcal{O}(N^2)$

DivideAndConquerDP.h

Description: Given $a[i] = \min_{lo(i) \leq k < hi(i)} (f(i, k))$ where the (minimal) optimal k increases with i , computes $a[i]$ for $i = L..R - 1$.

Time: $\mathcal{O}((N + (hi - lo)) \log N)$

d38d2b, 18 lines

```
242 struct DP { // Modify at will:
178     int lo(int ind) { return 0; }
072     int hi(int ind) { return ind; }
f99     ll f(int ind, int k) { return dp[ind][k]; }
55e     void store(int ind, int k, ll v) { res[ind] = pii(k, v); }

105     void rec(int L, int R, int LO, int HI) {
d2c         if (L >= R) return;
c52         int mid = (L + R) >> 1;
a4e         pair<ll, int> best (LLONG_MAX, LO);
964         rep(k, max(LO, lo(mid)), min(HI, hi(mid)))
af9             best = min(best, make_pair(f(mid, k), k));
4b0         store(mid, best.second, best.first);
ebc         rec(L, mid, LO, best.second+1);
ba2         rec(mid+1, R, best.second, HI);
541     }
26f     void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
d38 };
```

10.4 Debugging tricks

- signal(SIGSEGV, [](int) { _Exit(0); }); converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). _GLIBCXX_DEBUG failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).

- feenableexcept(29); kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16).

10.5 Optimization tricks

__builtin_ia32_ldmxcsr(40896); disables denormals (which make floats 20x slower near their minimum value).

10.5.1 Bit hacks

- x & -x is the least bit in x.

- for (int x = m; x;) { --x &= m; ... } loops over all subset masks of m (except m itself).

- c = x&-x, r = x+c; (((r^x) >> 2)/c) | r is the next number after x with the same number of bits set.

- rep(b,0,K) rep(i,0,(1 << K)) if (i & 1 << b) D[i] += D[i^(1 << b)]; computes all sums of subsets.

10.5.2 Pragmas

- #pragma GCC optimize ("Ofast") will make GCC auto-vectorize loops and optimizes floating points better.

- #pragma GCC target ("avx2") can double performance of vectorized code, but causes crashes on old machines.

- **#pragma** GCC optimize ("trapv") kills the program on integer overflows (but is really slow).

FastMod.h

Description: Compute $a\%b$ about 5 times faster than usual, where b is constant but not known at compile time. Returns a value congruent to a (mod b) in the range $[0,2b)$.

	751a02, 8 lines
f4c	typedef unsigned long long ull;
7e2	struct FastMod {
634	ull b, m;
d2d	FastMod(ull b) : b(b), m(-1ULL / b) {}
683	ull reduce(ull a) { <i>// a % b + (0 or b)</i>
6fa	return a - (ull)((_uint128_t(m) * a) >> 64) * b;
f67	}
38e	};

FastInput.h

Description: Read an integer from stdin. Usage requires your program to pipe in input from file.

Usage: ./a.out < input.txt

Time: About 5x as fast as cin/scanf.

e30	inline char gc() { <i>// like getchar()</i>	7b3c70, 17 lines
0cd	static char buf[1 << 16];	
0c8	static size_t bc, be;	
a5a	if (bc >= be) {	
bf4	buf[0] = 0, bc = 0;	
842	be = fread(buf, 1, sizeof (buf), stdin);	
d32	}	
efa	return buf[bc++]; <i>// returns 0 on EOF</i>	
026	}	

e4d	int readInt() {
db8	int a, c;
169	while ((a = gc()) < 40);
0cc	if (a == '-') return -readInt();
17e	while ((c = gc()) >= 48) a = a * 10 + c - 480;
24d	return a - 48;
e04	}

BumpAllocator.h

Description: When you need to dynamically allocate many objects and don't care about freeing them. "new X" otherwise has an overhead of something like 0.05us + 16 bytes per allocation.

	745db2, 8 lines
	<i>// Either globally or in a single class:</i>
2b9	static char buf[450 << 20];
a7c	void* operator new(size_t s) {
da1	static size_t i = sizeof buf;
3ca	assert(s < i);
663	return (void*)&buf[i -= s];
306	}
aa3	void operator delete(void*) {}

SmallPtr.h

Description: A 32-bit pointer that points into BumpAllocator memory.

	2dd6c9, 10 lines
0ca	template <class T> struct ptr {
949	unsigned ind;
185	ptr(T* p = 0) : ind(p ? unsigned ((char*)p - buf) : 0) {
3d4	assert(ind < sizeof buf);
77e	}
e3c	T& operator*(const { return *(T*)(buf + ind); };
570	T& operator->(const { return &***this; };
618	T& operator[](int a) const { return (&***this)[a]; };
e0a	explicit operator bool() const { return ind; };
2dd	};

BumpAllocatorSTL.h

Description: BumpAllocator for STL containers.

	bb66d4, 14 lines
	Usage: vector<vector<int, small<int>>> ed(N);
30c	char buf[450 << 20] alignas(16);
cee	size_t buf_ind = sizeof buf;
ebc	template <class T> struct small {
d7b	typedef T value_type;
36e	small() {}
1ca	template <class U> small(const U&) {}
de2	T* allocate(size_t n) {
207	buf_ind -= n * sizeof (T);
df0	buf_ind &= 0 - alignof(T);
d25	return (T*)(buf + buf_ind);
e76	}
e28	void deallocate(T*, size_t) {}
164	};

SIMD.h

Description: Cheat sheet of SSE/AVX intrinsics, for doing arithmetic on several numbers at once. Can provide a constant factor improvement of about 4, orthogonal to loop unrolling. Operations follow the pattern `__mm(256)?_name_(si(128|256)|epi(8|16|32|64)|pd|ps)".` Not all are described here; grep for `__mm_` in `/usr/lib/gcc/*/4.9/include/` for more. If AVX is unsupported, try 128-bit operations, "emmintrin.h" and `#define _SSE_` and `__MMX_` before including it. For aligned memory use `__mm_malloc(size, 32)` or `int buf[N] alignas(32)`, but prefer `loadu/storeu`.

ee8	#pragma GCC target ("avx2") <i>// or sse4.1</i>	551b82, 43 lines
492	#include "emmintrin.h"	
1b2	typedef __m256i mi;	
8ca	#define L(x) __mm256_loadu_si256((mi*)&(x))	
	<i>// High-level/specific methods:</i>	
	<i>// load(u)?_si256, store(u)?_si256, setzero_si256, __mm_malloc</i>	
	<i>// blendv_(epi8 ps pd) (z?y:x), movemask_epi8 (hibits of bytes)</i>	
	<i>// i32gather_epi32(addr, x, 4): map addr[] over 32-b parts of x</i>	
	<i>// sad_epu8: sum of absolute differences of u8, outputs 4xi64</i>	
	<i>// maddubs_epi16: dot product of unsigned i7's, outputs 16xi15</i>	
	<i>// madd_epi16: dot product of signed i16's, outputs 8xi32</i>	
	<i>// extractf128_si256(, i) (256->128), cvtsi128_si32 (128->lo32)</i>	
	<i>// permute2f128_si256(x,x,1) swaps 128-bit lanes</i>	
	<i>// shuffle_epi32(x, 3*64+2*16+1*4+0) == x for each lane</i>	
	<i>// shuffle_epi8(x, y) takes a vector instead of an __mm</i>	

// Methods that work with most data types (append e.g. _epi32):
// set1, blend (i8?x:y), add, adds (sat.), mullo, sub, and/or,
// andnot, abs, min, max, sign(1,x), cmp(gt|eq), unpack(lo|hi)

1e5	int sumi32(mi m) { union { int v[8]; mi m;} u; u.m = m;
6d0	int ret = 0; rep(i,0,8) ret += u.v[i]; return ret; }
49f	mi zero() { return __mm256_setzero_si256(); }
1e1	mi one() { return __mm256_set1_epi32(-1); }
667	bool all_zero(mi m) { return __mm256_testz_si256(m, m); }
382	bool all_one(mi m) { return __mm256_testc_si256(m, one()); }
rfo	ll example_filteredDotProduct(int n, short* a, short* b) {
f37	int i = 0; ll r = 0;
766	mi zero = __mm256_setzero_si256(), acc = zero;
fe1	while (i + 16 <= n) {
25c	mi va = L(a[i]), vb = L(b[i]); i += 16;
2a9	va = __mm256_and_si256(__mm256_cmpgt_epi16(vb, va), va);
9d0	mi vp = __mm256_madd_epi16(va, vb);
1ee	acc = __mm256_add_epi64(__mm256_unpacklo_epi32(vp, zero),
9d7	__mm256_add_epi64(acc, __mm256_unpackhi_epi32(vp, zero))
	);
b3a	}

Techniques (A)

techniques.txt

159 lines

Recursion

Divide and conquer

Finding interesting points in N log N

Algorithm analysis

Master theorem

Amortized time complexity

Greedy algorithm

Scheduling

Max contiguous subvector sum

Invariants

Huffman encoding

Graph theory

Dynamic graphs (extra book-keeping)

Breadth first search

Depth first search

* Normal trees / DFS trees

Dijkstra’s algorithm

MST: Prim’s algorithm

Bellman-Ford

Konig’s theorem and vertex cover

Min-cost max flow

Lovasz toggle

Matrix tree theorem

Maximal matching, general graphs

Hopcroft-Karp

Hall’s marriage theorem

Graphical sequences

Floyd-Warshall

Euler cycles

Flow networks

* Augmenting paths

* Edmonds-Karp

Bipartite matching

Min. path cover

Topological sorting

Strongly connected components

2-SAT

Cut vertices, cut-edges and biconnected components

Edge coloring

* Trees

Vertex coloring

* Bipartite graphs (=> trees)

* 3^n (special case of set cover)

Diameter and centroid

K’t h shortest path

Shortest cycle

Dynamic programming

Knapsack

Coin change

Longest common subsequence

Longest increasing subsequence

Number of paths in a dag

Shortest path in a dag

Dynprog over intervals

Dynprog over subsets

Dynprog over probabilities

Dynprog over trees

3^n set cover

Divide and conquer

Knuth optimization

Convex hull optimizations

RMQ (sparse table a.k.a 2^k-jumps)

Bitonic cycle

Log partitioning (loop over most restricted)

Combinatorics

Computation of binomial coefficients

Pigeon-hole principle

Inclusion/exclusion

Catalan number

Pick’s theorem

Number theory

Integer parts

Divisibility

Euclidean algorithm

Modular arithmetic

* Modular multiplication

* Modular inverses

* Modular exponentiation by squaring

Chinese remainder theorem

Fermat’s little theorem

Euler’s theorem

Phi function

Frobenius number

Quadratic reciprocity

Pollard-Rho

Miller-Rabin

Hensel lifting

Vieta root jumping

Game theory

Combinatorial games

Game trees

Mini-max

Nim

Games on graphs

Games on graphs with loops

Grundy numbers

Bipartite games without repetition

General games without repetition

Alpha-beta pruning

Probability theory

Optimization

Binary search

Ternary search

Unimodality and convex functions

Binary search on derivative

Numerical methods

Numeric integration

Newton’s method

Root-finding with binary/ternary search

Golden section search

Matrices

Gaussian elimination

Exponentiation by squaring

Sorting

Radix sort

Geometry

Coordinates and vectors

* Cross product

* Scalar product

Convex hull

Polygon cut

Closest pair

Coordinate-compression

Quadtrees

KD-trees

All segment-segment intersection

Sweeping

Discretization (convert to events and sweep)

Angle sweeping

Line sweeping

Discrete second derivatives

Strings

Longest common substring

Palindrome subsequences

Knuth-Morris-Pratt

Tries

Rolling polynomial hashes

Suffix array

Suffix tree

Aho-Corasick

Manacher’s algorithm

Letter position lists

Combinatorial search

Meet in the middle

Brute-force with pruning

Best-first (A*)

Bidirectional search

Iterative deepening DFS / A*

Data structures

LCA (2^k-jumps in trees in general)

Pull/push-technique on trees

Heavy-light decomposition

Centroid decomposition

Lazy propagation

Self-balancing trees

Convex hull trick (wcipeg.com/wiki/Convex_hull_trick)

Monotone queues / monotone stacks / sliding queues

Sliding queue using 2 stacks

Persistent segment tree